



## **DATA STRUCTURES & ALGORITHMS**

---





# Centre for Distance and Online Education

## Online BCA Program

### Data Structures & Algorithms

Semester: 4

---

#### Author

Dr. Bharsha Anjaria,  
Assistant Professor,  
Computer Science & Engineering,  
Parul institute of Technology,  
Parul University.

---

#### Credits

Centre for Distance and Online Education, Parul University,  
Post Limda, Waghodia,  
Vadodara, Gujarat, India  
391760.  
Website: <https://paruluniversity.ac.in/>

---

#### Disclaimer

This content is protected by CDOE, Parul University. It is sold under the stipulation that it cannot be lent, resold, hired out, or otherwise circulated without obtaining prior written consent from the publisher. The content should remain in the same binding or cover as it was initially published, and this requirement should also extend to any subsequent purchaser. Furthermore, it is important to note that, in compliance with the copyright protections outlined above, no part of this publication may be reproduced, stored in a retrieval system, or transmitted through any means (including electronic, Mechanical, photocopying, recording, or otherwise) without obtaining the prior written permission from both the copyright owner and the publisher of this content.

---



## Note to Students

These course notes are intended for the exclusive use of students enrolled in Online BCA Program. They are not to be shared or distributed without explicit permission from the University. Any unauthorized sharing or distribution of these materials may result in academic and legal consequences.

---



TABLE OF CONTENT

SUB LESSON 1.1..... 8

DATA VS INFORMATION ..... 8

SUB LESSON 1.2..... 12

WHAT IS DATA STRUCTURE ..... 12

SUB LESSON 1.3..... 15

TYPES OF DATA STRUCTURE ..... 15

SUB LESSON 1.4..... 20

ALGORITHM AND ITS PROPERTIES..... 20

SUB LESSON 1.5..... 25

EVALUATION OF ALGORITHM..... 25

SUB LESSON 2.1..... 31

WHAT IS SORTING AND TYPES OF SORTING ..... 31

SUB LESSON 2.2..... 35

BUBBLE SORT ..... 35

SUB LESSON 2.3..... 39

SELECTION SORT ..... 39

SUB LESSON 2.4..... 44

MERGE SORT ..... 44

SUB LESSON 2.5..... 50

QUICK SORT..... 50

SUB LESSON 3.1..... 57

SEARCHING IN DATA STRUCTURE ..... 57



SUB LESSON 3.2..... 61

LINEAR SEARCH ..... 61

SUB LESSON 3.3..... 65

BINARY SEARCH..... 65

SUB LESSON 4.1..... 71

INTRODUCTION TO STACK ..... 71

SUB LESSON 4.2..... 75

APPLICATION OF STACK - I ..... 75

SUB LESSON 4.3..... 82

APPLICATION IF STACK - II..... 82

SUB LESSON 4.4..... 89

APPLICATION OF STACK - III ..... 89

SUB LESSON 5.1..... 93

INTRODUCTION AND OPERATIONS ON QUEUE..... 93

SUB LESSON 5.2..... 98

TYPES OF QUEUE..... 98

SUB LESSON 5.3..... 102

APPLICATIONS OF QUEUE..... 102

SUB LESSON 6.1..... 106

INTRODUCTION AND TYPES OF LINKED LIST ..... 106

SUB LESSON 6.2..... 111

OPERATION ON SINGLY LINKED LIST ..... 111

SUB LESSON 6.3..... 121

OPERATION ON DOUBLY LINKED LIST..... 121



SUB LESSON 6.4..... 131

APPLICATION OF LINKED LIST ..... 131

SUB LESSON 7.1..... 135

BASIC TERMINOLOGIES OF TREE ..... 135

SUB LESSON 7.2..... 140

TYPES OF TREE ..... 140

SUB LESSON 7.3..... 144

OPERATION ON BINARY SEARCH TREE ..... 144

SUB LESSON 7.4..... 151

OPERATION ON AVL TREE ..... 151

SUB LESSON 8.1..... 161

BASIC TERMINOLOGIES OF GRAPH..... 161

SUB LESSON 8.2..... 165

TYPES OF GRAPH..... 165

SUB LESSON 8.3..... 170

BREADTH FIRST SEARCH ..... 170

SUB LESSON 8.4..... 174

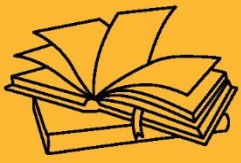
DEPTH FIRST SEARCH ..... 174

SUB LESSON 7.5..... 178

PRIMS ALGORITHM..... 178

SUB LESSON 7.6..... 183

KRUSKAL ALGORITHM..... 183



## INTRODUCTION TO DATA





## SUB LESSON 1.1

### DATA VS INFORMATION

#### OBJECTIVES OF CHAPTER 1

- Differentiate between data and information.
- Understand the concept and need for data structures.
- Explore various types of data structures.
- Learn what an algorithm is and its essential properties.
- Evaluate algorithm efficiency using time and space complexity.

Data and information are two related but distinct concepts in the area of computing and information processing.

Data refers to the raw, unprocessed facts and figures that are collected, stored, and manipulated by computers. These could be numbers, text, images, audio, or any other type of digital input. Data can be organized into various data structures, such as tables, graphs, or trees, depending on the type and format of the data. Data is a collection of raw facts, which may not provide much value or meaning.

On the other hand, information is the output or result of processing and analyzing data to derive meaning and insights. Information is the knowledge or understanding gained from data after it has been processed and organized in a meaningful way. It is the interpretation of the data, and it provides context and relevance to the raw facts and figures.

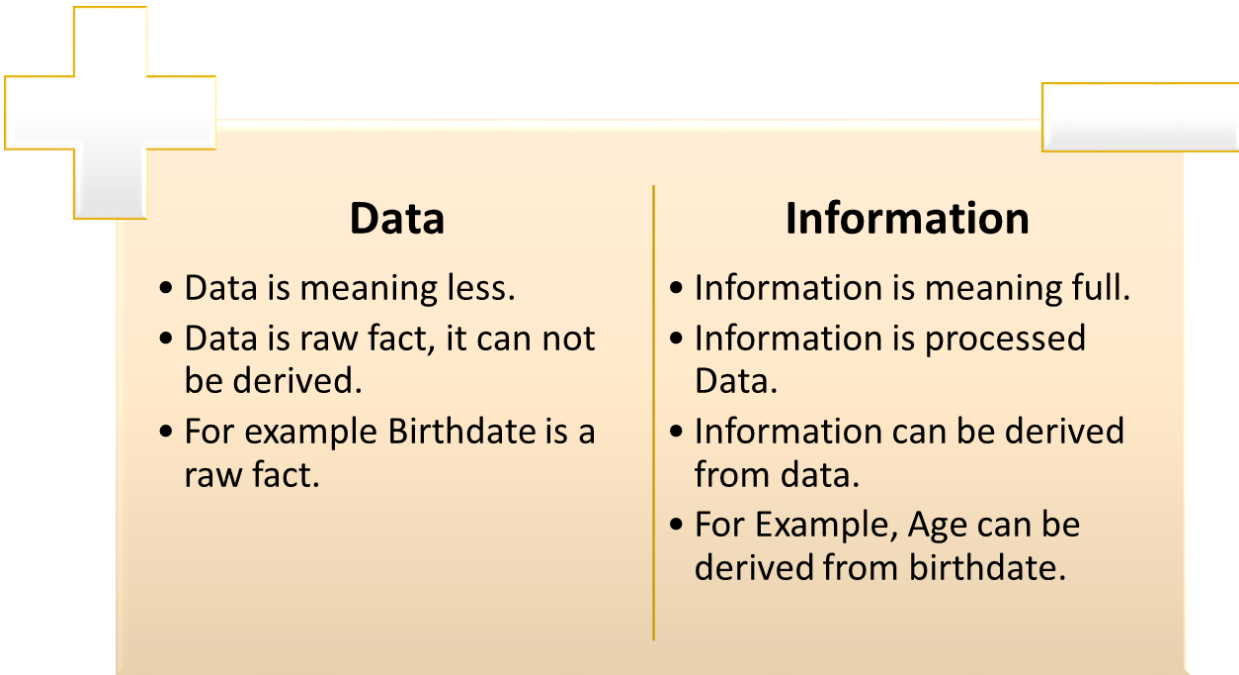
Information is derived from data by applying various methods of processing, such as sorting, filtering, categorizing, and analyzing. It is typically presented in a format that is understandable and useful to the intended audience, such as a report, graph, or chart. The way information is structured depends on the nature of the data being analyzed, the purpose of the analysis, and the preferences of the intended audience.





In terms of data structure, data can be organized into various formats, such as arrays, lists, and databases, to facilitate storage and manipulation. Information, on the other hand, is typically presented in a structured format that is easy to read and understand, such as tables, charts, or graphs. The choice of data structure for storing and processing data depends on the specific application and the type of analysis required to derive meaningful insights from it.

### DIFFERENCE BETWEEN DATA AND INFORMATION

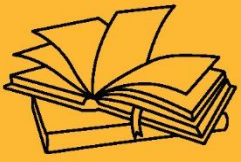




---

## KEY TAKEAWAYS

- Understanding the difference between data and information.
- Data refers to raw, unprocessed facts and figures, while information is processed, organized, and structured data that provides meaning and insights
- It's important to note that data can be structured, semi-structured, or unstructured, and that information is derived from data by applying various processing methods.
- Other topics related to data structure include algorithms, data types, data storage, and data manipulation techniques.
- if we focus only on the difference between data and information, we can understand the importance of organizing and processing data to derive valuable insights and make informed decision



## INTRODUCTION TO DATA





## SUB LESSON 1.2

### WHAT IS DATA STRUCTURE

#### WHAT IS DATA STRUCTURE

In computer science, a data structure is a way of organizing and storing data in a computer's memory so that it can be accessed and manipulated efficiently. The design and implementation of data structures are important topics in computer science and software engineering, as they can significantly impact the performance, memory usage, and complexity of computer programs.

Data structure refers to the way data is organized and stored in a computer's memory or disk storage. It is a fundamental concept in computer science and is used to efficiently store, retrieve, and manipulate data.

In simple terms, a data structure is a way of organizing and storing data so that it can be easily accessed and used efficiently. It provides a way to represent complex data in a logical and structured manner.

A data structure can be thought of as a container that holds data in a specific way. Different data structures are designed for different purposes and have different trade-offs in terms of their advantages and disadvantages. Some of the most common data structures used in computer science include arrays, linked lists, stacks, queues, trees, and graphs.

Arrays are a simple data structure that stores a collection of elements of the same type in contiguous memory locations. They provide fast access to individual elements but are less efficient when inserting or deleting elements in the middle of the array.

Linked lists are a data structure that stores elements in nodes that are linked together through pointers. They can be more efficient than arrays when inserting or deleting elements in the middle of the list, but may require more memory and have slower access times for individual elements.



Stacks and queues are data structures that allow elements to be added or removed only from one end of the structure. Stacks use a Last-In-First-Out (LIFO) policy, while queues use a First-In-First-Out (FIFO) policy.

Trees are a hierarchical data structure that organizes elements in a tree-like structure with a root node and child nodes. They are commonly used in algorithms such as binary search and heap sort.

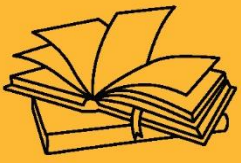
Graphs are a more complex data structure that represents a set of vertices or nodes connected by edges. They can be used to model relationships between objects, such as social networks or transportation networks.

The choice of data structure depends on the specific needs of the program, such as the type of data being stored, the operations that need to be performed on the data, and the performance requirements of the program. A well-designed data structure can greatly improve the efficiency, maintainability, and scalability of a program.

---

### KEY TAKEAWAYS

- The key takeaway of the basics of data structure is that it refers to the way data is organized and stored in a computer's memory or disk storage
- The primary goal of data structure is to provide an efficient way of accessing and manipulating data.
- Each data structure has its own strengths and weaknesses, and choosing the appropriate data structure depends on the specific use case and requirements.
- Understanding data structures is essential for computer programming and software development, as they provide a foundation for designing and implementing efficient algorithm
- By choosing the appropriate data structure and algorithm, programmers can optimize performance and improve the overall efficiency of their code.



## INTRODUCTION TO DATA





## SUB LESSON 1.3

### TYPES OF DATA STRUCTURE

#### TYPES OF STRUCTURE

Data structure refers to the way data is organized and stored in a computer's memory or disk storage. It is a fundamental concept in computer science and is used to efficiently store, retrieve, and manipulate data. In this article, we will provide a detailed description of the various types of data structures, their characteristics, and their applications.

#### LINEAR DATA STRUCTURES

Linear data structures are those in which the data elements are arranged in a sequential manner. The main advantage of linear data structures is that they are simple and easy to implement. Some examples of linear data structures include arrays, stacks, queues, and linked lists.

#### ARRAYS

Arrays are the simplest and most commonly used data structure. They consist of a collection of elements of the same data type, stored in contiguous memory locations. Arrays are used to store and manipulate lists of data, such as integers, characters, or strings. The main advantage of arrays is that they provide direct access to the elements, which allows for fast retrieval and manipulation of data. However, the size of an array is fixed at the time of creation, which can be a disadvantage if the size of the data is unknown or can change.

#### STACKS

Stacks are a type of data structure that follows the last-in, first-out (LIFO) principle. In other words, the last element added to the stack is the first one to be removed. Stacks are used to manage data that follows a specific order, such as undo/redo operations, backtracking, and parsing. Stacks are implemented using arrays or linked lists. The main operations on a stack are



push, which adds an element to the top of the stack, and pop, which removes the top element from the stack.

## QUEUES

Queues are a type of data structure that follows the first-in, first-out (FIFO) principle. In other words, the first element added to the queue is the first one to be removed. Queues are used to manage data that follows a specific order, such as scheduling, printing, and file I/O. Queues are implemented using arrays or linked lists. The main operations on a queue are enqueue, which adds an element to the back of the queue, and dequeue, which removes the front element from the queue.

## LINKED LISTS

Linked lists are a type of data structure that consists of a sequence of nodes, where each node contains a data element and a reference to the next node in the sequence. Linked lists are used to store and manipulate lists of data, such as integers, characters, or strings. The main advantage of linked lists is that they can dynamically grow or shrink in size, which makes them more flexible than arrays. However, linked lists have a higher overhead than arrays, and accessing elements is slower than in arrays.

Data structure refers to the way data is organized and stored in a computer's memory or disk storage. It is a fundamental concept in computer science and is used to efficiently store, retrieve, and manipulate data. In this article, we will provide a detailed description of the various types of data structures, their characteristics, and their applications.

## LINEAR DATA STRUCTURES

Linear data structures are those in which the data elements are arranged in a sequential manner. The main advantage of linear data structures is that they are simple and easy to implement. Some examples of linear data structures include arrays, stacks, queues, and linked lists.

## TREES





Trees are a type of data structure that consists of a collection of nodes, where each node contains a data element and a reference to its child nodes. Trees are used to represent hierarchical relationships between data elements, such as file systems, XML documents, and organization charts. Trees have a single root node, from which all other nodes descend. The main advantage of trees is that they provide a hierarchical structure that allows for efficient searching, insertion, and deletion of data. However, the complexity of tree operations can be higher than other data structures, and the structure of the tree can affect the efficiency of operations.

## GRAPHS

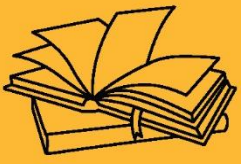
Graphs are a type of data structure that consists of a collection of nodes, where each node represents a data element, and a collection of edges, where each edge represents a relationship between two nodes. Graphs are used to represent complex relationships between data elements, such as social networks, road networks, and chemical compounds. Graphs can be directed or undirected, weighted or unweighted, and cyclic or acyclic. The main advantage of graphs is that they provide a flexible way of representing complex relationships between data elements. However, the complexity of graph operations can be higher than other data structures, and the structure of the graph can affect



---

## KEY TAKEAWAYS

- The key takeaway of the basics of data structure is that it refers to the way data is organized and stored in a computer's memory or disk storage
- The primary goal of data structure is to provide an efficient way of accessing and manipulating data.
- Each data structure has its own strengths and weaknesses, and choosing the appropriate data structure depends on the specific use case and requirements.
- Understanding data structures is essential for computer programming and software development, as they provide a foundation for designing and implementing efficient algorithm
- By choosing the appropriate data structure and algorithm, programmers can optimize performance and improve the overall efficiency of their code.



## INTRODUCTION TO DATA





## SUB LESSON 1.4

### ALGORITHM AND ITS PROPERTIES

#### ALGORITHM AND ITS PROPERTIES

Algorithms are step-by-step procedures or instructions for solving a specific problem. They serve as a blueprint or roadmap for performing a task or achieving a desired outcome. In computer science, algorithms are at the core of software development, enabling efficient problem-solving and data manipulation.

Design characteristics of algorithms refer to the key principles and qualities that guide their development. These characteristics ensure that algorithms are effective, efficient, and maintainable. In this section, we will delve into the definition of algorithms and explore their design characteristics in detail.

#### DEFINITION AND PURPOSE OF ALGORITHMS

An algorithm is a precise set of instructions that takes an input and produces a desired output through a series of well-defined steps. It is a computational procedure that solves a specific problem or performs a particular task. Algorithms can be implemented in various programming languages and are crucial for automating processes, making decisions, and performing computations.

The purpose of algorithms is to solve problems efficiently and reliably. They enable complex tasks to be broken down into smaller, more manageable steps. By providing a systematic approach to problem-solving, algorithms allow for consistent and repeatable results. They are utilized in a wide range of applications, including data analysis, artificial intelligence, optimization, cryptography, and more.

#### PROPERTIES OF ALGORITHMS

##### CORRECTNESS



The correctness of an algorithm refers to its ability to produce the correct output for any given input. An algorithm must solve the problem it is designed for accurately and precisely. To ensure correctness, algorithms must be thoroughly tested and validated using different input scenarios, including normal cases, edge cases, and boundary cases. Formal methods such as mathematical proofs and formal verification techniques can be employed to establish the correctness of an algorithm.

### EFFICIENCY

Efficiency is a crucial design characteristic of algorithms, focusing on their ability to solve problems in a timely manner. Efficient algorithms utilize resources optimally, minimizing the amount of time, memory, and computational power required to solve a problem. Efficiency is typically measured by analyzing the time complexity and space complexity of an algorithm. Time complexity evaluates the amount of time required to execute an algorithm, while space complexity assesses the amount of memory or storage space used.

### SCALABILITY

Scalability refers to an algorithm's ability to handle larger problem sizes without a significant decrease in efficiency. As the input size grows, an algorithm should not experience a disproportionate increase in execution time or resource usage. Scalable algorithms are crucial for handling real-world scenarios where data sizes can be extensive. Designing algorithms that exhibit good scalability often involves considering data structures and algorithmic techniques that can efficiently handle large datasets.

### MAINTAINABILITY

Maintainability pertains to the ease with which an algorithm can be understood, modified, and adapted over time. Well-designed algorithms are structured, modular, and utilize clear and concise code. They employ best practices, adhere to coding standards, and document their logic and assumptions effectively. Maintaining algorithms becomes essential when new requirements



arise, bugs need fixing, or enhancements are necessary. A maintainable algorithm allows developers to make changes without introducing unintended side effects.

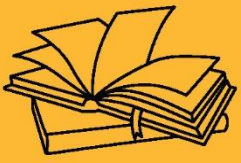
## REUSABILITY

Reusability refers to the ability to reuse or adapt an algorithm for solving similar or related problems. Designing algorithms that are modular and abstracted allows them to be easily repurposed or combined with other algorithms to solve different problems. Reusability reduces redundant code, promotes code organization, and facilitates code maintenance. By creating reusable algorithms, developers can save time and effort by building upon existing solutions rather than reinventing the wheel.



## KEY TAKEAWAYS

- **Algorithm:** An algorithm is a precise set of instructions that solves a specific problem or performs a task. It provides a systematic approach for solving problems and automating processes in computer science.
- **Design Characteristics of Algorithms:**
  - a. **Correctness:** Algorithms should produce the correct output for any given input, validated through testing and formal methods.
  - b. **Efficiency:** Algorithms should solve problems in a timely manner, utilizing resources optimally. Time complexity and space complexity are analyzed to measure efficiency.
  - c. **Scalability:** Algorithms should handle larger problem sizes without a significant decrease in efficiency, allowing for the processing of large datasets.
  - d. **Maintainability:** Algorithms should be structured, modular, and well-documented, making them easy to understand, modify, and adapt over time.
  - e. **Reusability:** Algorithms should be designed with modularity and abstraction, enabling their reuse and adaptation for solving similar or related problems.
- By incorporating these design characteristics into algorithm development, developers can create reliable, efficient, scalable, maintainable, and reusable solutions.
- Algorithms play a vital role in various domains of computer science and are essential for problem-solving and data manipulation.
- Understanding their design principles allows for the development of effective algorithms and the efficient resolution of complex problems.



## INTRODUCTION TO DATA







## SUB LESSON 1.5

### EVALUATION OF ALGORITHM

#### EVALUATION OF ALGORITHM

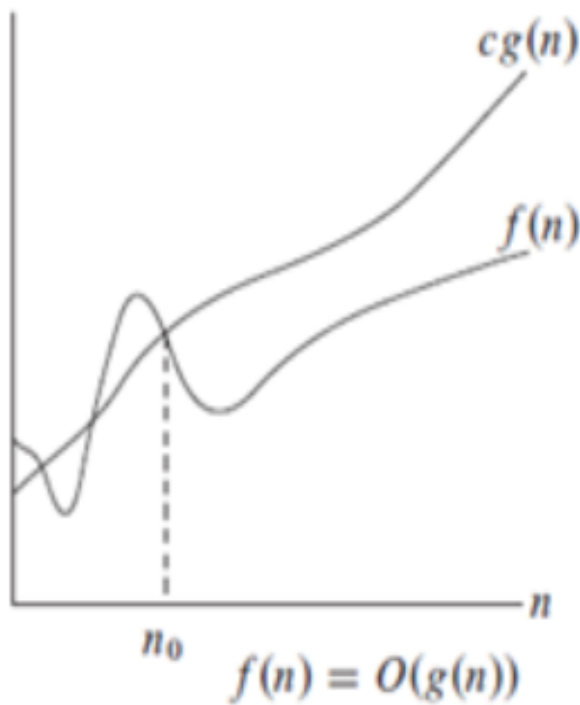
Asymptotic notation is used in algorithm analysis to describe the growth rate of algorithms in terms of their input size. There are three commonly used asymptotic notations: Big O notation, Omega notation, and Theta notation. Each notation provides different information about the behavior of an algorithm as the input size increases.

#### BIG O NOTATION (O-NOTATION):

Big O notation describes the upper bound or worst-case scenario of the growth rate of an algorithm. It represents an algorithm's maximum possible time or space complexity. In simpler terms, it gives an upper limit on the resources (time or space) required by an algorithm.

Mathematically, we denote an algorithm's time complexity using Big O notation as  $O(f(n))$ , where " $f(n)$ " represents a function that characterizes the algorithm's growth rate. The Big O notation emphasizes the worst-case scenario, which means that the algorithm will not perform worse than this upper bound.

Example: Suppose we have an algorithm with a time complexity of  $O(n^2)$ . This means that the algorithm's time requirements grow quadratically with the input size ( $n$ ). As the input size increases, the algorithm's execution time increases at a rate proportional to the square of the input size.

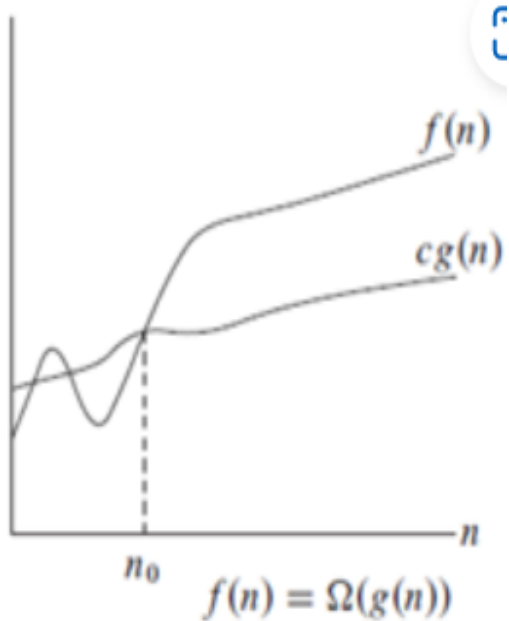


#### OMEGA NOTATION ( $\Omega$ -NOTATION):

Omega notation represents the lower bound or best-case scenario of an algorithm's growth rate. It provides information about the minimum amount of resources (time or space) required by an algorithm to solve a problem.

Mathematically, Omega notation is denoted as  $\Omega(g(n))$ , where " $g(n)$ " represents a function that characterizes the algorithm's lower bound growth rate. The Omega notation emphasizes the best-case scenario, which means that the algorithm will not perform better than this lower bound.

Example: If an algorithm has a time complexity of  $\Omega(n)$ , it means that the algorithm's time requirements grow linearly with the input size ( $n$ ). As the input size increases, the algorithm's execution time will increase at a rate proportional to the input size.

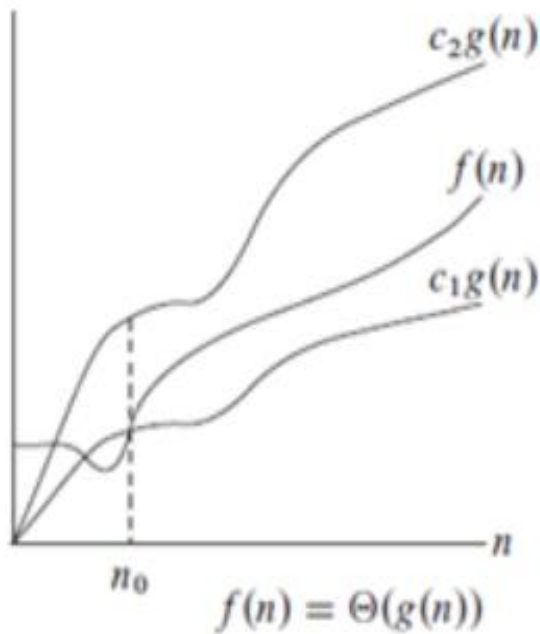


#### THETA NOTATION ( $\Theta$ -NOTATION):

Theta notation provides a tight bound on an algorithm's growth rate by combining both the upper and lower bounds. It defines the average-case scenario of an algorithm's growth rate, considering both the best and worst cases.

Mathematically, Theta notation is denoted as  $\Theta(h(n))$ , where " $h(n)$ " represents a function that characterizes the algorithm's growth rate, and it satisfies both the upper and lower bounds.

Example: If an algorithm has a time complexity of  $\Theta(n)$ , it means that the algorithm's time requirements grow linearly with the input size ( $n$ ), and it has the same order of growth in the best and worst cases.



### KEY TAKEAWAYS

- Asymptotic notation is used to analyze the performance of algorithms by describing their growth rate as the input size increases.
- Big O notation (O-notation) provides an upper bound or worst-case scenario for an algorithm's time or space complexity. It represents the maximum resources required by the algorithm.
- Omega notation ( $\Omega$ -notation) gives a lower bound or best-case scenario for an algorithm's growth rate. It represents the minimum resources required by the algorithm.
- Theta notation ( $\Theta$ -notation) provides a tight bound on an algorithm's growth rate. It represents the average-case scenario where the algorithm's performance falls within a specific range.
- Big O notation emphasizes the upper limit on an algorithm's growth rate, Omega notation emphasizes the lower limit, and Theta notation provides both upper and lower bounds.

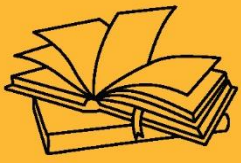


- Asymptotic notations are useful for comparing the efficiency of algorithms, identifying the best and worst-case scenarios, and making informed decisions in algorithm design and selection.
- Graphically, Big O notation is represented as an upper bound line, Omega notation is represented as a lower bound line, and Theta notation represents a range between the upper and lower bounds.
- Understanding asymptotic notation enables developers and analysts to evaluate the scalability and efficiency of algorithms and choose the most appropriate algorithm for specific problem-solving tasks.
- It aids in estimating the resource requirements of algorithms and making informed decisions regarding algorithm design and optimization.

---

## OUTCOMES OF CHAPTER 1

- Clearly distinguish between data and information.
- Explain the role and importance of data structures in computing.
- Identify and describe various data structure types (linear, non-linear).
- Define an algorithm and assess its key properties.
- Evaluate algorithm efficiency through Big O notation and complexity analysis.
- Understand how to choose appropriate data structures based on problem requirements.



## **OPERATIONS ON DATA STRUCTURE**

---





## SUB LESSON 2.1

### WHAT IS SORTING AND TYPES OF SORTING

#### OBJECTIVES OF CHAPTER 2

- Understand the purpose and significance of sorting in data handling.
- Learn different sorting algorithms: Bubble, Selection, Merge, and Quick Sort.
- Analyze the time and space complexity of each sorting method.
- Compare the performance of sorting techniques based on input types.
- Understand real-world applications of sorting.

Sorting is the process of arranging a collection of elements or records in a specific order, typically in ascending or descending order, based on certain criteria. It is an essential operation in computer science and data structures, as it allows efficient searching, insertion, and retrieval of data.

There are various types of sorting algorithms, which differ in their efficiency, implementation, and performance. Some of the most commonly used sorting algorithms include:

#### BUBBLE SORT:

Bubble sort repeatedly compares adjacent elements and swaps them if they are in the wrong order. It continues iterating through the entire collection until the elements are sorted. It has a time complexity of  $O(n^2)$  in the average and worst cases.

#### SELECTION SORT:

Selection sort divides the collection into two parts: the sorted part and the unsorted part. It repeatedly selects the smallest element from the unsorted part and moves it to the sorted part. It has a time complexity of  $O(n^2)$  in all cases.

#### INSERTION SORT:



Insertion sort builds the final sorted array one element at a time. It takes an element from the unsorted part and inserts it into the correct position in the sorted part. It has a time complexity of  $O(n^2)$  in the average and worst cases.

#### MERGE SORT:

Merge sort follows the divide-and-conquer approach. It recursively divides the collection into two halves, sorts them independently, and then merges them to obtain the final sorted array. It has a time complexity of  $O(n \log n)$  in all cases, making it more efficient than the previous algorithms for large collections.

#### QUICK SORT:

Quick sort also uses the divide-and-conquer approach. It selects a pivot element and partitions the collection into two sub-arrays, one with elements less than the pivot and the other with elements greater than the pivot. It then recursively sorts the sub-arrays. Quick sort has an average time complexity of  $O(n \log n)$ , but its worst-case time complexity is  $O(n^2)$  if the pivot selection is not optimized.

#### HEAP SORT:

Heap sort constructs a binary heap from the collection and repeatedly extracts the maximum element from the heap, which results in a sorted array. It has a time complexity of  $O(n \log n)$  in all cases.

#### RADIX SORT:

Radix sort sorts the elements by processing individual digits or characters. It starts with the least significant digit and moves towards the most significant digit, repeatedly partitioning the elements based on each digit. Radix sort has a time complexity of  $O(k * n)$ , where  $k$  is the maximum number of digits or characters in the elements.



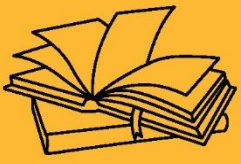


These are just a few examples of sorting algorithms, and there are many more, each with its own strengths and weaknesses. The choice of sorting algorithm depends on factors such as the size of the collection, the distribution of data, and the available resources.

---

## KEY TAKEAWAYS

- **Efficiency:** Sorting algorithms can have different time complexities, which determine their efficiency. It's important to consider the size of the data set and the expected performance requirements when choosing a sorting algorithm.
- **Stability:** Some sorting algorithms maintain the relative order of elements with equal keys, while others may not. Stability is important when dealing with data where the order of equal elements matters.
- **In-place vs. Out-of-place:** In-place sorting algorithms modify the original array without requiring additional memory, while out-of-place algorithms create a separate copy of the array. The choice between these depends on memory constraints and the need to preserve the original order.
- **Adaptiveness:** Adaptive sorting algorithms perform well on partially sorted or nearly sorted data, as they can take advantage of existing order. Non-adaptive algorithms perform the same number of operations regardless of the initial order.
- **Recursion vs. Iteration:** Sorting algorithms can be implemented using either recursive or iterative approaches. Recursive algorithms divide the problem into smaller subproblems, while iterative algorithms use loops to iteratively process the data.
- **Practical Considerations:** Besides theoretical efficiency, practical considerations such as ease of implementation, code complexity, and available library functions can influence the choice of a sorting algorithm.



## **OPERATIONS ON DATA STRUCTURE**

---





## SUB LESSON 2.2

### BUBBLE SORT

#### BUBBLE SORT

Bubble sort is a simple sorting algorithm that repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. The algorithm continues this process until the entire list is sorted.

Here's an example to illustrate how bubble sort works:

Consider an array of integers: [5, 2, 8, 12, 1]

#### PASS 1:

Compare 5 and 2: Swap them. [2, 5, 8, 12, 1]

Compare 5 and 8: No swap. [2, 5, 8, 12, 1]

Compare 8 and 12: No swap. [2, 5, 8, 12, 1]

Compare 12 and 1: Swap them. [2, 5, 8, 1, 12]

#### PASS 2:

Compare 2 and 5: No swap. [2, 5, 8, 1, 12]

Compare 5 and 8: No swap. [2, 5, 8, 1, 12]

Compare 8 and 1: Swap them. [2, 5, 1, 8, 12]

#### PASS 3:

Compare 2 and 5: No swap. [2, 5, 1, 8, 12]

Compare 5 and 1: Swap them. [2, 1, 5, 8, 12]



#### PASS 4:

Compare 2 and 1: Swap them. [1, 2, 5, 8, 12]

After four passes, the array is sorted in ascending order: [1, 2, 5, 8, 12].

In the worst case, when the array is in reverse order, bubble sort requires multiple passes to sort the array. In each pass, it compares and swaps adjacent elements until the largest element "bubbles" up to its correct position.

For an array of size  $n$ , bubble sort will perform  $(n-1)$  passes in the worst case. In each pass, it compares  $(n-1)$ ,  $(n-2)$ ,  $(n-3)$ , ..., 2, and 1 pairs of adjacent elements. Therefore, the total number of comparisons in the worst case can be calculated as:

$$(n-1) + (n-2) + (n-3) + \dots + 2 + 1$$

$$= (n*(n-1))/2$$

This sum represents the number of comparisons made by bubble sort. Since the number of swaps is typically similar to the number of comparisons, we can conclude that the worst-case time complexity of bubble sort is  $O(n^2)$ .

Here's a C program that implements bubble sort:

```
#include <stdio.h>

void bubbleSort(int arr[], int size) {
    int i, j, temp;
    for (i = 0; i < size - 1; i++) {
        for (j = 0; j < size - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                // Swap elements
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
```



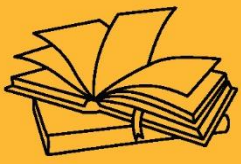
```
        arr[j + 1] = temp;
    }
}
}
}

int main() {
    int arr[] = {5, 2, 8, 12, 1};
    int size
```

---

### KEY TAKEAWAYS

- Bubble sort is a simple sorting algorithm that iteratively compares adjacent elements and swaps them if they are in the wrong order.
- It continues this process until the entire list is sorted, with the largest element "bubbling" to the end of the list in each pass.
- Bubble sort has a time complexity of  $O(n^2)$  in the worst case, where  $n$  is the number of elements in the list.
- It is an in-place sorting algorithm, meaning it does not require additional memory beyond the original array.
- Bubble sort is not the most efficient sorting algorithm, particularly for large datasets. It is often outperformed by more advanced sorting algorithms like merge sort or quick sort.
- Despite its simplicity, bubble sort can be useful for educational purposes or for sorting small datasets where simplicity and ease of implementation are more important than efficiency.
- Bubble sort is stable, meaning it preserves the relative order of elements with equal keys.
- It can be optimized by adding a flag to check if any swaps were made in a pass, allowing for early termination if the list is already sorted.



## **OPERATIONS ON DATA STRUCTURE**

---



## SUB LESSON 2.3

### SELECTION SORT

#### SELECTION SORT

Selection sort is a simple sorting algorithm that repeatedly selects the minimum (or maximum) element from an unsorted part of the array and places it at the beginning (or end) of the sorted part. The algorithm continues this process until the entire array is sorted.

Here's an example to illustrate how selection sort works:

Consider an array of integers: [5, 2, 8, 12, 1]

#### PASS 1:

Find the minimum element in the unsorted part (1, 2, 8, 12, 5). The minimum is 1.

Swap the minimum element (1) with the first element of the unsorted part (5). [1, 2, 8, 12, 5]

#### PASS 2:

Find the minimum element in the unsorted part (2, 8, 12, 5). The minimum is 2.

Swap the minimum element (2) with the second element of the unsorted part (8). [1, 2, 8, 12, 5]

#### PASS 3:

Find the minimum element in the unsorted part (8, 12, 5). The minimum is 5.

Swap the minimum element (5) with the third element of the unsorted part (8). [1, 2, 5, 12, 8]

#### PASS 4:

Find the minimum element in the unsorted part (12, 8). The minimum is 8.

Swap the minimum element (8) with the fourth element of the unsorted part (12). [1, 2, 5, 8, 12]



After four passes, the array is sorted in ascending order: [1, 2, 5, 8, 12].

Now, let's derive the time complexity of selection sort. In each pass, selection sort finds the minimum (or maximum) element from the unsorted part of the array, requiring  $(n-1)$ ,  $(n-2)$ ,  $(n-3)$ , ..., 2, and 1 comparisons for an array of size  $n$ .

The total number of comparisons in the worst case can be calculated as:

$$(n-1) + (n-2) + (n-3) + \dots + 2 + 1$$

$$= (n*(n-1))/2$$

This sum represents the number of comparisons made by selection sort. Since the number of swaps is typically similar to the number of comparisons, we can conclude that the worst-case time complexity of selection sort is  $O(n^2)$ .

Here's a C program that implements selection sort:

```
#include <stdio.h>

void selectionSort(int arr[], int size) {
    int i, j, minIndex, temp;
    for (i = 0; i < size - 1; i++) {
        minIndex = i;
        // Find the index of the minimum element in the unsorted part
        for (j = i + 1; j < size; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        // Swap the minimum element with the current element
        temp = arr[i];
        arr[i] = arr[minIndex];
    }
}
```





```
        arr[minIndex] = temp;
    }
}

int main() {
    int arr[] = {5, 2, 8, 12, 1};
    int size = sizeof(arr) / sizeof(arr[0]);
    printf("Original array: ");
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

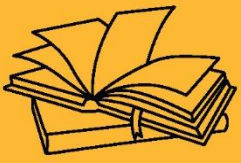
    selectionSort(arr, size);
    printf("Sorted array: ");
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
    return 0;
}
```



---

## KEY TAKEAWAYS

- Selection sort is a simple sorting algorithm that repeatedly selects the minimum (or maximum) element from an unsorted part of the array and places it at the beginning (or end) of the sorted part.
- It continues this process until the entire array is sorted.
- Selection sort has a time complexity of  $O(n^2)$  in the worst case, where  $n$  is the number of elements in the array.
- It is an in-place sorting algorithm, meaning it does not require additional memory beyond the original array.
- Selection sort is not the most efficient sorting algorithm, particularly for large datasets. It performs more swaps than other sorting algorithms such as insertion sort or quick sort.
- Despite its simplicity, selection sort can be useful for educational purposes or for sorting small datasets where simplicity and ease of implementation are more important than efficiency.
- Selection sort is not stable, meaning it may change the relative order of elements with equal keys.
- It is a comparison-based sorting algorithm, meaning it relies on comparing elements to determine their order.
- Selection sort divides the array into two parts: the sorted part and the unsorted part. In each iteration, it expands the sorted part by selecting the minimum (or maximum) element from the unsorted part and placing it in its correct position in the sorted part.



## **OPERATIONS ON DATA STRUCTURE**





## SUB LESSON 2.4

### MERGE SORT

#### MERGE SORT

Merge sort is a divide-and-conquer algorithm that works by dividing the input array into smaller subarrays, sorting them recursively, and then merging them back together to obtain a sorted array.

Here's an example to illustrate how merge sort works:

Consider an array of integers: [5, 2, 8, 12, 1]

Step 1: Divide the array into smaller subarrays until each subarray contains a single element.

[5, 2, 8, 12, 1] is divided into [5, 2], [8], [12], [1].

Step 2: Merge the subarrays by comparing and arranging their elements in sorted order.

Merge [5, 2] and [8] to obtain [2, 5, 8].

Merge [12] and [1] to obtain [1, 12].

Merge [2, 5, 8] and [1, 12] to obtain the sorted array [1, 2, 5, 8, 12].

After the merging step, the array is sorted in ascending order: [1, 2, 5, 8, 12].

Now, let's derive the time complexity of merge sort. The algorithm divides the array into smaller subarrays in a divide step, and then merges the subarrays in a conquer step. The time complexity can be derived as follows:

Divide step: The array is recursively divided into halves until each subarray contains a single element. This process takes  $O(\log n)$  time, where  $n$  is the number of elements in the array.

Conquer step: Merging the subarrays back together to obtain the sorted array takes  $O(n)$  time, as each element is visited once during the merging process.



Since the divide and conquer steps are performed independently, the overall time complexity of merge sort is  $O(n \log n)$ .

Here's a C program that implements merge sort:

```
#include <stdio.h>
```

```
void merge(int arr[], int left[], int leftSize, int right[], int rightSize) {
```

```
    int i = 0, j = 0, k = 0;
```

```
    while (i < leftSize && j < rightSize) {
```

```
        if (left[i] <= right[j]) {
```

```
            arr[k] = left[i];
```

```
            i++;
```

```
        } else {
```

```
            arr[k] = right[j];
```

```
            j++;
```

```
        }
```

```
        k++;
```

```
    }
```

```
    while (i < leftSize) {
```

```
        arr[k] = left[i];
```

```
        i++;
```

```
        k++;
```

```
    }
```



```
while (j < rightSize) {  
  
    arr[k] = right[j];  
  
    j++;  
  
    k++;  
  
}  
  
}  
  
void mergeSort(int arr[], int size) {  
  
    if (size < 2) {  
  
        return;  
  
    }  
  
    int mid = size / 2;  
  
    int left[mid];  
  
    int right[size - mid];  
  
    for (int i = 0; i < mid; i++) {  
  
        left[i] = arr[i];  
  
    }  
  
    for (int i = mid; i < size; i++) {  
  
        right[i - mid] = arr[i];  
  
    }  
  
    mergeSort(left, mid);
```



```
mergeSort(right, size - mid);

merge(arr, left, mid, right, size - mid);

}

int main() {

    int arr[] = {5, 2, 8, 12, 1};

    int size = sizeof(arr) / sizeof(arr[0]);

    printf("Original array: ");

    for (int i = 0; i < size; i++) {

        printf("%d ", arr[i]);

    }

    printf("\n");

    mergeSort(arr, size);

    printf("Sorted array: ");

    for (int i = 0; i < size; i++) {

        printf("%d ", arr[i]);

    }

    printf("\n");

    return 0;

}
```

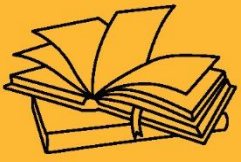


---

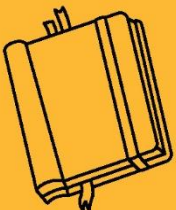
## KEY TAKEAWAYS

- Merge sort is a divide-and-conquer algorithm that works by recursively dividing the input array into smaller subarrays, sorting them, and then merging them back together to obtain a sorted array.
- It has a time complexity of  $O(n \log n)$  in the worst case, where  $n$  is the number of elements in the array.
- Merge sort is a stable sorting algorithm, meaning it preserves the relative order of elements with equal keys.
- It is an out-of-place sorting algorithm, meaning it requires additional memory to store the merged subarrays during the merging process.
- Merge sort is suitable for sorting large datasets and is particularly efficient for sorting linked lists due to its efficient merging process.
- The algorithm guarantees sorting even in the worst-case scenario and performs well on both small and large datasets.
- Merge sort is based on the principle of divide and conquer, where the problem is divided into smaller sub-problems that are solved independently and then combined to obtain the final solution.
- It divides the array into halves until each subarray contains a single element and then merges the subarrays by comparing and arranging their elements in sorted order.
- Merge sort is widely used in practice due to its efficiency and stability, making it a popular choice for sorting tasks in programming and computer science.





## **OPERATIONS ON DATA STRUCTURE**





## SUB LESSON 2.5

### QUICK SORT

#### QUICK SORT

Quick sort is a divide-and-conquer algorithm that works by selecting a pivot element from the array and partitioning the other elements into two sub-arrays, according to whether they are less than or greater than the pivot. The sub-arrays are then sorted recursively. This process is repeated until the entire array is sorted.

Here's an example to illustrate how quick sort works:

Consider an array of integers: [5, 2, 8, 12, 1]

Step 1: Choose a pivot element. Let's select the last element, 1, as the pivot.

Step 2: Partition the array by rearranging the elements such that all elements less than the pivot are to its left, and all elements greater than the pivot are to its right. After partitioning, we have [1, 2, 5, 12, 8].

Step 3: Recursively apply quick sort to the two sub-arrays formed on either side of the pivot.

For the left sub-array [1, 2], we select 2 as the pivot, and after partitioning, we get [1, 2].

For the right sub-array [12, 8], we select 8 as the pivot, and after partitioning, we get [8, 12].

Step 4: The sub-arrays are now sorted, and we combine them to obtain the sorted array: [1, 2, 5, 8, 12].

After the partitioning and recursive sorting steps, the array is sorted in ascending order: [1, 2, 5, 8, 12].

Now, let's derive the time complexity of quick sort. The time complexity depends on the choice of the pivot and the partitioning process.



In the best case scenario, where the pivot always divides the array into two equal-sized sub-arrays, the time complexity of quick sort is  $O(n \log n)$ , where  $n$  is the number of elements in the array.

In the worst case scenario, where the pivot is always the smallest or largest element, the partitioning process only separates one element at a time. In this case, the time complexity is  $O(n^2)$ .

However, on average, quick sort has a time complexity of  $O(n \log n)$ . This is due to the efficient partitioning process that divides the array into sub-arrays of roughly equal size.

Quick sort is an in-place sorting algorithm, meaning it does not require additional memory beyond the original array. It is also not a stable sorting algorithm, meaning it may change the relative order of elements with equal keys.

In summary, quick sort is an efficient sorting algorithm with an average time complexity of  $O(n \log n)$ . It is widely used in practice due to its simplicity and good average-case performance. The choice of the pivot and the partitioning process play a crucial role in the efficiency of quick sort.

Here's a C program that implements quick sort:

```
#include <stdio.h>
```

```
void swap(int* a, int* b) {
```

```
    int temp = *a;
```

```
    *a = *b;
```

```
    *b = temp;
```

```
}
```



```
int partition(int arr[], int low, int high) {
```

```
    int pivot = arr[high];
```

```
    int i = low - 1;
```

```
    for (int j = low; j <= high - 1; j++) {
```

```
        if (arr[j] <= pivot) {
```

```
            i++;
```

```
            swap(&arr[i], &arr[j]);
```

```
        }
```

```
    }
```

```
    swap(&arr[i + 1], &arr[high]);
```

```
    return i + 1;
```

```
}
```

```
void quickSort(int arr[], int low, int high) {
```

```
    if (low < high) {
```

```
        int pi = partition(arr, low, high);
```

```
        quickSort(arr, low, pi - 1);
```

```
        quickSort(arr, pi + 1, high);
```

```
    }
```

```
}
```

```
int main() {
```



```
int arr[] = {5, 2, 8, 12, 1};

int size = sizeof(arr) / sizeof(arr[0]);

printf("Original array: ");

for (int i = 0; i < size; i++) {

    printf("%d ", arr[i]);

}

printf("\n");

quickSort(arr, 0, size - 1);

printf("Sorted array: ");

for (int i = 0; i < size; i++) {

    printf("%d ", arr[i]);

}

printf("\n");

return 0;

}
```



---

## KEY TAKEAWAYS

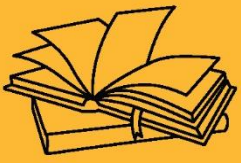
- Quick sort is a divide-and-conquer algorithm that works by selecting a pivot element and partitioning the array into two sub-arrays, one with elements smaller than the pivot and the other with elements greater than the pivot.
- The pivot element is placed in its final sorted position, and the process is recursively applied to the sub-arrays until the entire array is sorted.
- Quick sort has an average time complexity of  $O(n \log n)$  and a worst-case time complexity of  $O(n^2)$ . However, the worst-case scenario is rare and occurs when the pivot selection is unfavorable.
- It is an in-place sorting algorithm, meaning it sorts the array without requiring additional memory beyond the original array.
- Quick sort is not a stable sorting algorithm, meaning it may change the relative order of elements with equal keys.
- The efficiency of quick sort heavily depends on the choice of the pivot element. A good pivot selection strategy can improve the average case performance.
- Quick sort is widely used in practice due to its average-case efficiency and simplicity of implementation.
- The algorithm has a recursive nature and uses the divide-and-conquer technique, which involves dividing the problem into smaller sub-problems, solving them independently, and combining the solutions to obtain the final result.
- Quick sort can be further optimized using various techniques such as randomizing the pivot selection, using median-of-three as the pivot, or switching to an alternative sorting algorithm like insertion sort for small subarrays.
- It is suitable for large datasets and performs well in practice when implemented efficiently.



---

## OUTCOME OF CHAPTER 2

- Describe the purpose and need for sorting data.
- Implement common sorting algorithms: Bubble, Selection, Merge, Quick Sort.
- Analyze and compare the performance (time/space complexity) of sorting methods.
- Select the most efficient sorting algorithm for a given dataset.
- Gain practical skills in coding and debugging sorting routines.
- Understand the role of sorting in optimizing other data processing tasks.



## **OPERATIONS ON DATA STRUCTURE**

---







## SUB LESSON 3.1

### SEARCHING IN DATA STRUCTURE

#### OBJECTIVES OF CHAPTER 3

- Introduce searching and its importance in data processing.
- Learn linear and binary search algorithms.
- Understand the differences between sequential and binary search.
- Analyze the efficiency of each searching method.
- Identify suitable use cases for each search technique.

Searching is a fundamental operation in data structures and plays a crucial role in various applications. The importance of searching lies in the ability to efficiently retrieve specific data elements from a collection based on their values. Let's discuss the significance of searching in data structures, focusing on binary search and linear search, along with their complexities.

#### LINEAR SEARCH:

Linear search is a simple search algorithm that sequentially checks each element in a data structure until the desired element is found or the entire collection has been traversed.

#### IMPORTANCE:

Linear search is applicable to any data structure that allows sequential access, such as arrays, linked lists, or unsorted lists.

It is straightforward to implement and does not require any specific ordering or structure.

#### COMPLEXITY:

Worst-case time complexity:  $O(n)$ , where  $n$  is the number of elements in the data structure. In the worst case, the search operation may need to examine all elements until finding the target or reaching the end.



Average-case time complexity:  $O(n/2)$  or  $O(n)$ , as the average case assumes the target element is equally likely to be found at any position.

#### **BINARY SEARCH:**

Binary search is a more efficient search algorithm applicable to sorted data structures. It repeatedly divides the search space in half, narrowing down the search range based on comparisons with the middle element.

#### **IMPORTANCE:**

Binary search is particularly useful when working with large sorted arrays or lists where efficiency is critical.

It dramatically reduces the number of comparisons required compared to linear search, especially for large data sets.

Binary search is commonly used as a building block for other search algorithms and data structures.

#### **COMPLEXITY:**

Worst-case time complexity:  $O(\log n)$ , where  $n$  is the number of elements in the sorted data structure. In each comparison step, the search space is halved, resulting in a logarithmic time complexity.

Average-case time complexity:  $O(\log n)$ . In average cases, binary search performs significantly better than linear search due to its logarithmic time complexity.

---

#### **COMPARING THE COMPLEXITIES:**

Binary search has a better time complexity than linear search, especially for large data sets, as it reduces the search space by half at each step.

Linear search has a worst-case time complexity of  $O(n)$ , while binary search has a worst-case time complexity of  $O(\log n)$ . This means that binary search can be much faster for large collections.



However, it's important to note that binary search requires the data structure to be sorted, which may involve additional overhead if the data needs frequent updates.

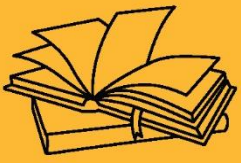
Linear search is simpler to implement and does not require any specific ordering or structure.

In summary, the importance of searching in data structures lies in the ability to efficiently locate specific elements. Linear search is versatile and applicable to any data structure, while binary search provides significant performance improvements when working with sorted data structures. The choice between the two depends on factors such as the size of the data set, presence of sorting, and the trade-off between simplicity and efficiency.

---

### KEY TAKEAWAYS

- **Importance:** Searching is a fundamental operation in data structures and plays a crucial role in various applications. It enables efficient retrieval of specific data elements based on their values.
- **Time Complexity:** The time complexity of a search algorithm determines its efficiency. Different data structures and search algorithms have different time complexities, which impact the speed of the search operation. Understanding the time complexity helps in choosing the most suitable data structure and algorithm for a given problem.
- **Sorted vs. Unsorted Data:** The efficiency of search algorithms varies depending on whether the data is sorted or unsorted. Sorted data allows for more efficient search algorithms like binary search, while unsorted data may require linear search or other techniques.
- **Trade-offs:** Different search algorithms and data structures have trade-offs between efficiency and additional overheads such as sorting requirements or memory usage. It's important to consider these trade-offs based on the specific requirements of the problem at hand.
- **Versatility:** Searching is applicable to a wide range of data structures, including arrays, linked lists, trees, hash tables, and more. Different data structures have different characteristics and are suitable for different scenarios.



## **OPERATIONS ON DATA STRUCTURE**

---





## SUB LESSON 3.2

### LINEAR SEARCH

#### LINEAR SEARCH

Linear search, also known as sequential search, is a basic search algorithm used to find a specific element within a collection of data. It sequentially checks each element in the data structure until either the target element is found or the entire collection has been traversed.

Here's how the linear search algorithm works:

- Start from the beginning of the data structure.
- Compare the current element with the target element.
- If the current element matches the target element, the search is successful, and the algorithm returns the index or position of the element.
- If the current element does not match the target element, move to the next element in the data structure.
- Repeat steps 2-4 until either the target element is found or the end of the data structure is reached.
- If the end of the data structure is reached without finding the target element, the algorithm concludes that the element is not present.

Linear search is applicable to various data structures, including arrays, linked lists, and unsorted lists. It does not require any specific ordering or structure within the data.

The time complexity of linear search is  $O(n)$ , where  $n$  is the number of elements in the data structure. In the worst case, the algorithm may need to examine all elements until finding the target or reaching the end. The average-case time complexity is  $O(n/2)$  or simply  $O(n)$ , assuming that the target element is equally likely to be found at any position.

Linear search is simple to implement and works well for small data sets or when the data is unordered. However, it becomes inefficient for large data sets or when performance is a critical



factor. In such cases, more advanced search algorithms like binary search or hash-based search structures (e.g., hash tables) provide better efficiency.

Let's say we have an array of integers: [5, 2, 8, 4, 9, 3, 1] and we want to search for the element 9 using linear search.

Start from the beginning of the array.

- Compare the first element, 5, with the target element, 9. They are not equal, so move to the next element.
- Compare the second element, 2, with the target element, 9. They are not equal, so move to the next element.
- Continue this process until we find the target element or reach the end of the array.
- Compare the sixth element, 3, with the target element, 9. They are not equal, so move to the next element.
- Compare the last element, 1, with the target element, 9. They are not equal, and we have reached the end of the array.
- Since we have traversed the entire array and have not found the target element, we conclude that the element 9 is not present in the array.

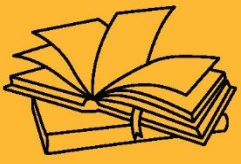
In this example, the linear search required checking each element in the array until the target element was found or the end of the array was reached. The time complexity of linear search is  $O(n)$ , where  $n$  is the number of elements in the array.



---

## KEY TAKEAWAYS

- **Sequential Process:** Linear search involves sequentially checking each element in a collection until the target element is found or the end of the collection is reached.
- **Applicability:** Linear search can be applied to various data structures, including arrays, linked lists, and unsorted lists. It does not require any specific ordering or structure within the data.
- **Time Complexity:** The worst-case time complexity of linear search is  $O(n)$ , where  $n$  is the number of elements in the data structure. The algorithm may need to examine all elements in the worst case.
- **Average Case:** In the average case, linear search examines approximately half of the elements, resulting in an average-case time complexity of  $O(n/2)$  or simply  $O(n)$ .
- **Simplicity:** Linear search is easy to understand and implement, making it suitable for small data sets or situations where simplicity is prioritized over performance.
- **Inefficiency for Large Data Sets:** Linear search becomes inefficient for large data sets, especially when compared to more advanced search algorithms like binary search or hash-based search structures.
- **Unordered Data:** Linear search is effective for searching unsorted or unordered data, as it does not rely on any specific ordering or structure.
- **Remember that the choice of search algorithm depends on various factors, such as the size of the data set, the presence of sorting or structure, and the desired performance trade-offs. Linear search is a basic search algorithm that serves as a foundation for understanding more complex searching techniques.**



## **OPERATIONS ON DATA STRUCTURE**

---







## SUB LESSON 3.3

### BINARY SEARCH

#### BINARY SEARCH

Binary search is a widely used searching algorithm that efficiently locates a target element within a sorted collection of data. It follows a divide-and-conquer strategy, repeatedly dividing the search space in half until the target element is found or determined to be absent.

Here's how binary search works:

- Start with a sorted collection of data, typically an array.
- Initialize two pointers, low and high, pointing to the first and last elements of the collection, respectively.
- Calculate the middle index as mid using the formula:  $\text{mid} = (\text{low} + \text{high}) / 2$ .
- Compare the target element with the element at the middle index.
- If the target element matches the middle element, the search is successful, and the algorithm returns the index or position of the element.
- If the target element is less than the middle element, update  $\text{high} = \text{mid} - 1$  to search in the lower half of the collection.
- If the target element is greater than the middle element, update  $\text{low} = \text{mid} + 1$  to search in the upper half of the collection.
- Repeat steps 3-7 until the target element is found or the search space is exhausted (i.e., low becomes greater than high).
- If the search space is exhausted and the target element is not found, the algorithm concludes that the element is not present in the collection.

Key properties and considerations of binary search:



- **Sorted Data:** Binary search requires the data to be sorted in ascending or descending order. If the data is unsorted, a sorting step must be performed before applying binary search.
- **Time Complexity:** The time complexity of binary search is  $O(\log n)$ , where  $n$  is the number of elements in the collection. At each step, the search space is divided in half, resulting in a logarithmic time complexity.
- **Efficiency:** Binary search is highly efficient for large collections. It drastically reduces the number of comparisons required compared to linear search, making it a preferred choice for sorted data sets.
- **Iterative or Recursive Implementation:** Binary search can be implemented using either an iterative or recursive approach. Both approaches follow the same fundamental principles.
- **Unique Elements:** Binary search assumes that the collection contains unique elements. If duplicate elements are present, the algorithm may not find the first occurrence of the target element.
- **Middle Index Calculation:** The middle index calculation can be modified based on programming language or specific implementation requirements to avoid potential overflow issues.

Binary search is commonly used in various applications, including searching in databases, efficiently locating elements in sorted arrays, and determining insertion points in ordered collections.

Overall, binary search provides an efficient and effective method for searching in sorted collections, offering significant time savings compared to linear search.

Consider the following sorted array of integers: [2, 5, 7, 10, 14, 18, 21, 27]. We want to search for the element 10 using binary search.

Start with the entire sorted array.



- Set the lower bound low to the first index of the array (0) and the upper bound high to the last index of the array (7).
- Calculate the middle index as  $\text{mid} = (\text{low} + \text{high}) / 2$ , which gives us  $\text{mid} = (0 + 7) / 2 = 3$ .
- Compare the target element 10 with the element at the middle index ( $\text{array}[3] = 10$ ).
- Since the target element matches the middle element, the search is successful, and we can conclude that the target element 10 is found at index 3.

In this example, binary search eliminates half of the search space at each step. After comparing the target element with the middle element, we determine whether to search in the lower or upper half of the array. By continuously dividing the search space in half, binary search efficiently finds the target element.

If we were searching for the element 21 in the same array using binary search, the steps would be as follows:

- Start with the entire sorted array.
- Set  $\text{low} = 0$  and  $\text{high} = 7$ .
- Calculate  $\text{mid} = (\text{low} + \text{high}) / 2 = (0 + 7) / 2 = 3$ .
- Compare the target element 21 with the element at the middle index ( $\text{array}[3] = 10$ ).
- Since the target element (21) is greater than the middle element (10), update  $\text{low} = \text{mid} + 1$  to search in the upper half of the array.
- Repeat the steps with the updated low and high values.
- Calculate  $\text{mid} = (\text{low} + \text{high}) / 2 = (4 + 7) / 2 = 5$ .
- Compare the target element 21 with the element at the middle index ( $\text{array}[5] = 18$ ).
- Since the target element (21) is greater than the middle element (18), update  $\text{low} = \text{mid} + 1$  to search in the upper half of the remaining search space.
- Repeat the steps with the updated low and high values.
- Calculate  $\text{mid} = (\text{low} + \text{high}) / 2 = (6 + 7) / 2 = 6$ .
- Compare the target element 21 with the element at the middle index ( $\text{array}[6] = 21$ ).



- The target element matches the middle element, and we conclude that the target element 21 is found at index 6.

Binary search significantly reduces the search space at each step, making it highly efficient for large sorted collections. It can find the target element in logarithmic time complexity, resulting in faster searches compared to linear search.

---

## KEY TAKEAWAYS

- **Sorted Data:** Binary search requires the data to be sorted in ascending or descending order. If the data is unsorted, a sorting step must be performed before applying binary search.
- **Divide-and-Conquer:** Binary search follows a divide-and-conquer strategy, dividing the search space in half at each step. This approach efficiently reduces the search space, leading to faster searches.
- **Time Complexity:** The time complexity of binary search is  $O(\log n)$ , where  $n$  is the number of elements in the collection. At each step, the search space is divided in half, resulting in a logarithmic time complexity.
- **Efficiency:** Binary search is highly efficient for large collections. It drastically reduces the number of comparisons required compared to linear search, making it a preferred choice for sorted data sets.
- **Iterative or Recursive Implementation:** Binary search can be implemented using either an iterative or recursive approach. Both approaches follow the same fundamental principles.
- **Unique Elements:** Binary search assumes that the collection contains unique elements. If duplicate elements are present, the algorithm may not find the first occurrence of the target element.
- **Versatility:** Binary search can be applied to various data structures, including arrays, lists, and trees, as long as the data is sorted.



- Middle Index Calculation: The middle index calculation can be modified based on programming language or specific implementation requirements to avoid potential overflow issues.
- Fast Retrieval: Binary search allows for fast retrieval of elements in sorted collections. It can quickly determine the presence or absence of a target element and provide its index or position.

---

### OUTCOMES OF CHAPTER 3

- Understand the significance of searching in data management.
- Implement linear and binary search algorithms effectively.
- Analyze the best, average, and worst-case performance of each search method.
- Apply suitable search strategies based on data structure and size.
- Develop problem-solving skills using efficient searching techniques.
- Gain proficiency in writing and testing search algorithms in code.



## **SIMPLE LINEAR DATA STRUCTURE**





## SUB LESSON 4.1

### INTRODUCTION TO STACK

#### OBJECTIVES OF CHAPTER 4

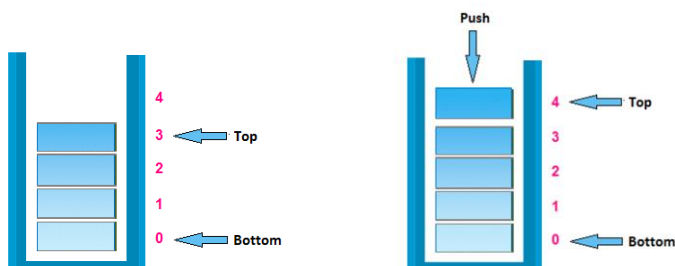
- Understand the concept and operations of a stack (LIFO).
- Explore implementation techniques using arrays and linked lists.
- Study real-world applications such as expression evaluation and recursion.
- Solve problems using stack-based logic.
- Implement and trace stack operations in programs.

A stack is a linear data structure that organizes data in a sequential manner. It operates on the principle of Last-In-First-Out (LIFO), which means that the last element added to the stack will be the first one to be removed. This makes it a suitable choice when you need to keep track of the most recently added elements.

Stack Operations:

Push:

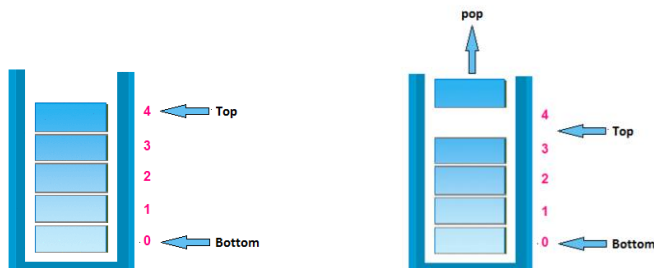
The push operation adds an element to the top of the stack. It takes the new element and places it at the top, making it the most recently added element. As a result, the new element becomes the new top of the stack.



Pop:

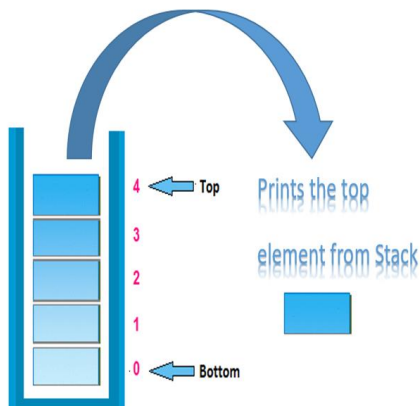


The pop operation removes the top element from the stack. It removes the element that was most recently added, exposing the element that was added before it. After the removal, the previous top element is now the new top of the stack.



Peek (or Top):

The peek operation allows you to examine the top element of the stack without removing it. It returns the value of the top element, providing access to the most recently added data.



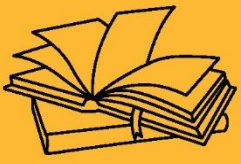




---

## KEY TAKEAWAYS

- Stack is a linear data structure that follows the Last-In-First-Out (LIFO) principle.
- The stack is organized as a sequential collection of elements, where the most recently added element is at the top.
- The main operations on a stack are:
  - Push: Adds an element to the top of the stack.
  - Pop: Removes the top element from the stack.
  - Peek (or Top): Allows you to examine the top element without removing it.
- Stack operations can be implemented using different data structures, such as arrays or linked lists.
- Stack data structure is commonly used in various applications, such as expression evaluation, function calls, undo/redo functionality, and backtracking algorithms.
- Understanding the stack and its operations is essential for solving problems that require LIFO behavior and managing data in a last-in-first-out order.



## **SIMPLE LINEAR DATA STRUCTURE**





## SUB LESSON 4.2

### APPLICATION OF STACK – I

#### APPLICATION OF STACK - 1

Stacks have various applications in computer science and software development due to their Last-In-First-Out (LIFO) nature. Here are some common applications of stacks:

Infix, Prefix, Postfix Notation :

In computer science and mathematics, expressions can be represented using different notations, such as infix, prefix, and postfix. These notations determine the placement of operators and operands within an expression. Here's an explanation of each notation:

Infix Notation:

Infix notation is the most common way of representing mathematical expressions, where operators are written between the operands. For example:

Arithmetic expression:  $2 + 3 * 4$

Algebraic expression:  $(a + b) / (c - d)$

Infix notation follows the conventional order of precedence for operators. Parentheses can be used to override the default precedence.

Prefix Notation (also known as Polish Notation):

Prefix notation places operators before their operands. This notation eliminates the need for parentheses or operator precedence rules. For example:

Arithmetic expression:  $+ 2 * 3 4$

Algebraic expression:  $/ + a b - c d$



In prefix notation, operators are applied to the next two operands to the right. The expression is evaluated from right to left.

Postfix Notation (also known as Reverse Polish Notation or RPN):

Postfix notation places operators after their operands. Like prefix notation, postfix notation doesn't require parentheses or operator precedence rules. For example:

Arithmetic expression:  $2\ 3\ 4\ *\ +$

Algebraic expression:  $a\ b\ +\ c\ d\ -\ /$

In postfix notation, operators are applied to the previous two operands to the left. The expression is evaluated from left to right.

Prefix and postfix notations are often used in stack-based computer architectures because they can be evaluated more easily by a computer program.

Infix to postfix conversion:

Converting an infix expression to postfix (or postfix) notation is often done using the shunting-yard algorithm. This algorithm makes use of a stack to rearrange the operators and operands in the correct order. Here's the step-by-step process of converting an infix expression to postfix notation:

1. Initialize an empty stack to store operators temporarily.
2. Scan the infix expression from left to right.
3. For each element encountered (operand or operator), perform the following actions:
  - a. If it is an operand (a number or variable), output it directly to the postfix expression.
  - b. If it is an opening parenthesis '(', push it onto the stack.
  - c. If it is a closing parenthesis ')', pop operators from the stack and output them to the postfix expression until an opening parenthesis is encountered. Discard the opening parenthesis.



d. If it is an operator:

- i. While there are operators already on the stack and the topmost operator has higher precedence than the current operator (or has the same precedence and is left-associative), pop the operator from the stack and output it to the postfix expression.
- ii. Push the current operator onto the stack.

Note: Precedence refers to the priority of operators, and associativity determines how operators of the same precedence are grouped.

4. Once all the elements have been scanned, pop any remaining operators from the stack and output them to the postfix expression.

5. The resulting postfix expression is the desired conversion.

Let's take an example to illustrate the process. We will convert the infix expression " $2 + 3 * 4$ " to postfix notation:

Step 1: Empty stack, empty output:

Stack:

Output:

Step 2: Scan the infix expression from left to right:

Element: 2

Stack:

Output: 2

Element: +

Stack: +

Output: 2

Element: 3

Stack: +

Output: 2 3

Element: \*

Stack: + \*



Output: 2 3

Element: 4

Stack: + \*

Output: 2 3 4

Step 4: Pop the remaining operators from the stack and output them:

Stack: +

Output: 2 3 4 \*

Final postfix expression: 2 3 4 \* +

The infix expression "2 + 3 \* 4" is converted to postfix notation as "2 3 4 \* +".

Infix to prefix conversion:

Converting an infix expression to prefix notation requires a similar approach to the infix to postfix conversion. We can use the reverse of the input infix expression and modify the shunting-yard algorithm to achieve this. Here's the step-by-step process of converting an infix expression to prefix notation:

1. Reverse the input infix expression.
2. Initialize an empty stack to store operators temporarily.
3. Scan the reversed infix expression from left to right.
4. For each element encountered (operand or operator), perform the following actions:
  - a. If it is an operand (a number or variable), output it directly to the prefix expression.
  - b. If it is an opening parenthesis '(', push it onto the stack.
  - c. If it is a closing parenthesis ')', pop operators from the stack and output them to the prefix expression until an opening parenthesis is encountered. Discard the opening parenthesis.
  - d. If it is an operator:
    - i. While there are operators already on the stack and the topmost operator has higher precedence than the current operator (or has the same precedence and is left-associative), pop the operator from the stack and output it to the prefix expression.



ii. Push the current operator onto the stack.

1. Note: Precedence refers to the priority of operators, and associativity determines how operators of the same precedence are grouped.

5. Once all the elements have been scanned, pop any remaining operators from the stack and output them to the prefix expression.

6. Reverse the resulting prefix expression to obtain the final converted expression.

Let's take an example to illustrate the process. We will convert the infix expression " $2 + 3 * 4$ " to prefix notation:

Step 1: Reverse the infix expression: " $4 * 3 + 2$ "

Step 2: Empty stack, empty output:

Stack:

Output:

Step 3: Scan the reversed infix expression from left to right:

Element: 4

Stack:

Output: 4

Element: \*

Stack: \*

Output: 4

Element: 3

Stack: \*

Output: 4 3

Element: +

Stack: + \*

Output: 4 3

Element: 2

Stack: + \*



Output: 4 3 2

Step 5: Pop the remaining operators from the stack and output them:

Stack:

Output: \* + 4 3 2

Final prefix expression: \* + 4 3 2

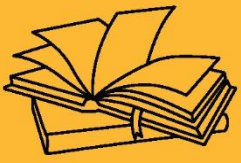
By following these steps, reversing the input, and modifying the shunting-yard algorithm, you can convert any infix expression to prefix notation. Remember to reverse the resulting prefix expression to obtain the final converted expression.

---

### KEY TAKEAWAYS

- Infix notation is commonly used in human-readable mathematical expressions and programming languages.
- Prefix and postfix notations are often used in stack-based computer architectures and for efficient expression evaluation.
- Stacks play a crucial role in converting infix expressions to prefix or postfix notations.
- The shunting-yard algorithm is commonly used for converting infix expressions to postfix or prefix notations.
- Overall, the choice of notation depends on the specific requirements of a given application. Infix notation is generally easier for humans to read and write, while prefix and postfix notations are often used for efficient evaluation and processing by computers.





## **SIMPLE LINEAR DATA STRUCTURE**

---



## SUB LESSON 4.3

### APPLICATION IF STACK – II

#### APPLICATION IF STACK - II

##### EVALUATION OF PREFIX NOTATION:

Evaluating a prefix notation expression involves applying operators to the corresponding operands from right to left. Here's the step-by-step process to evaluate a prefix expression:

Start with the given prefix expression.

Scan the expression from right to left.

For each element encountered, perform the following actions:

If it is an operand (number or variable), push it onto the stack.

If it is an operator, pop the top two elements from the stack as operands.

Apply the operator to the operands, with the top element popped considered as the left operand and the next popped element as the right operand.

Push the result back onto the stack.

Repeat steps 3 until all elements in the expression have been processed.

The final result is the value left on the stack.

Let's work through an example to illustrate the evaluation of a prefix expression: "+ \* 4 3 2"

Step 1: Start with the given prefix expression: "+ \* 4 3 2"

Step 2: Scan the expression from right to left.

Step 3: Perform the following actions:



Element: 2

Stack: 2

Element: 3

Stack: 3 2

Element: 4

Stack: 4 3 2

Element: \*

Pop: 4 3

Result: 12

Stack: 12 2

Element: +

Pop: 12 2

Result: 14

Step 4: All elements have been processed.

Step 5: The final result is 14.

Therefore, the evaluation of the prefix expression "+ \* 4 3 2" yields a result of 14.

Evaluation of postfix notation:

Evaluating a postfix notation expression involves applying operators to the corresponding operands from left to right. Here's the step-by-step process to evaluate a postfix expression:

Start with the given postfix expression.

Initialize an empty stack to store operands.

Scan the expression from left to right.

For each element encountered, perform the following actions:

If it is an operand (number or variable), push it onto the stack.



If it is an operator, pop the top two elements from the stack as operands.

Apply the operator to the operands, with the second popped element considered as the left operand and the first popped element as the right operand.

Push the result back onto the stack.

Repeat steps 4 until all elements in the expression have been processed.

The final result is the value left on the stack.

Let's work through an example to illustrate the evaluation of a postfix expression: "4 3 \* 2 +"

Step 1: Start with the given postfix expression: "4 3 \* 2 +"

Step 2: Initialize an empty stack.

Step 3: Scan the expression from left to right.

Step 4: Perform the following actions:

Element: 4

Stack: 4

Element: 3

Stack: 3 4

Element: \*

Pop: 3 4

Result: 12

Stack: 12

Element: 2

Stack: 2 12

Element: +

Pop: 2 12

Result: 14



Step 5: All elements have been processed.

Step 6: The final result is 14.

Therefore, the evaluation of the postfix expression " $4\ 3\ *\ 2\ +$ " yields a result of 14.

Tower of Hanoi :

The Tower of Hanoi is a classic mathematical puzzle that involves three pegs and a set of disks of different sizes. The goal is to move all the disks from the starting peg to another peg, following specific rules:

Only one disk can be moved at a time.

A larger disk cannot be placed on top of a smaller disk.

All disks must be moved to the destination peg.

Let's consider the Tower of Hanoi problem with three disks, labeled as A, B, and C, and three pegs, labeled as 1, 2, and 3. Here's the step-by-step process to solve the problem:

Step 1: Initial setup

Start with all the disks stacked in ascending order of size on the first peg (1). The largest disk (Disk 3) is at the bottom, followed by Disk 2, and Disk 1 on top.

Step 2: Move Disk 1

Move the smallest disk (Disk 1) from the first peg (1) to the second peg (2).

Step 3: Move Disk 2

Move the next larger disk (Disk 2) from the first peg (1) to the third peg (3).

Step 4: Move Disk 1

Move Disk 1 from the second peg (2) to the third peg (3).

Step 5: Move Disk 3



Move the largest disk (Disk 3) from the first peg (1) to the second peg (2).

Step 6: Move Disk 1

Move Disk 1 from the third peg (3) to the first peg (1).

Step 7: Move Disk 2

Move Disk 2 from the third peg (3) to the second peg (2).

Step 8: Move Disk 1

Move Disk 1 from the first peg (1) to the second peg (2).

After following these steps, all three disks will be successfully moved from the starting peg to the destination peg, satisfying the rules of the Tower of Hanoi puzzle.

The Tower of Hanoi problem has a recursive solution. As the number of disks increases, the number of moves required grows exponentially, following the formula  $2^n - 1$ , where  $n$  is the number of disks.



---

## KEY TAKEAWAYS

- Prefix notation (also known as Polish notation) and postfix notation (also known as Reverse Polish notation) are alternative ways of representing mathematical expressions.
- Evaluating prefix and postfix expressions involves applying operators to the corresponding operands.
- Prefix evaluation starts from the rightmost element, while postfix evaluation starts from the leftmost element.
- Both prefix and postfix notations eliminate the need for parentheses and operator precedence rules.
- Stack-based algorithms are commonly used to evaluate prefix and postfix expressions.
- Prefix and postfix notations are commonly used in stack-based computer architectures and for efficient expression evaluation.
- The Tower of Hanoi is a classic mathematical puzzle that involves moving a stack of disks from one peg to another.
- The puzzle consists of three pegs and a set of disks of different sizes.
- The objective is to move all the disks to another peg, following specific rules: only one disk can be moved at a time, larger disks cannot be placed on top of smaller disks, and all disks must be moved to the destination peg.
- The Tower of Hanoi problem has a recursive solution.
- The number of moves required to solve the Tower of Hanoi problem grows exponentially with the number of disks, following the formula  $2^n - 1$ , where  $n$  is the number of disks.



## **SIMPLE LINEAR DATA STRUCTURE**

---







## SUB LESSON 4.4

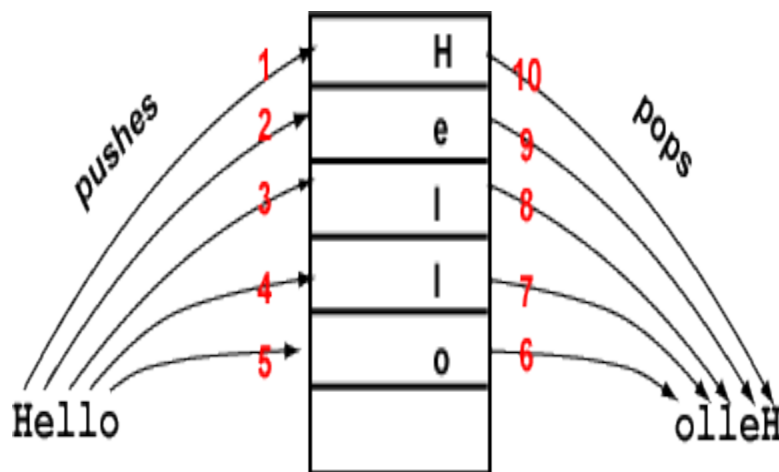
### APPLICATION OF STACK – III

#### APPLICATION OF STACK - III

##### REVERSING A STRING:

Reversing a string using a stack is a common approach that utilizes the Last-In-First-Out (LIFO) property of stacks. Here's the step-by-step process to reverse a string using a stack:

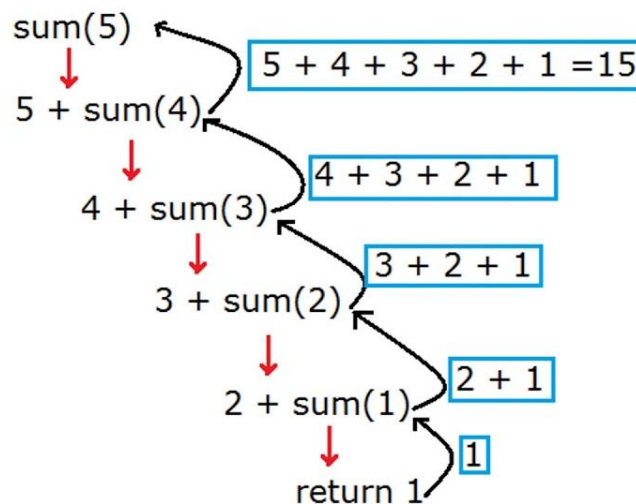
1. Create an empty stack.
2. Iterate through each character in the input string from left to right.
3. Push each character onto the stack.
4. After iterating through all the characters, the stack will contain the characters of the string in reverse order.
5. Pop each character from the stack and append it to a new string (or print directly if desired).
6. The resulting string will be the reversed form of the original string.



By pushing each character onto the stack and then popping them in reverse order, the original string is effectively reversed.

## RECURSION:

Recursion and stacks are closely related, and recursion can be seen as an application of a stack data structure. When a function is called recursively, the system maintains a stack called the call stack to keep track of the nested function calls. Each recursive call is added to the top of the call stack, and when a recursive call is completed, it is removed from the stack, allowing the system to return to the previous recursive call.



Recursion is a powerful concept that can simplify complex problems by breaking them down into smaller sub problems. The use of the call stack allows the system to manage the nested function calls and track the state of each recursive call. However, it's important to ensure that recursion is used appropriately and that termination conditions and base cases are properly defined to avoid infinite recursion and stack overflow errors.



---

## KEY TAKEAWAYS

- Reversing a string using a stack involves pushing each character onto a stack and then popping them in reverse order to obtain the reversed string.
- The Last-In-First-Out (LIFO) property of a stack makes it suitable for reversing the order of elements.
- The process includes iterating through the original string, pushing each character onto the stack, and then popping them one by one to construct the reversed string.
- Reversing a string using a stack can be done iteratively by manually managing a stack or by utilizing built-in stack data structures in programming languages.
- The time complexity of reversing a string using a stack is  $O(n)$ , where  $n$  is the length of the string
- Reversing a string using a stack involves pushing each character onto a stack and then popping them in reverse order to obtain the reversed string.
- The Last-In-First-Out (LIFO) property of a stack makes it suitable for reversing the order of elements.
- The process includes iterating through the original string, pushing each character onto the stack, and then popping them one by one to construct the reversed string.
- Reversing a string using a stack can be done iteratively by manually managing a stack or by utilizing built-in stack data structures in programming languages.
- The time complexity of reversing a string using a stack is  $O(n)$ , where  $n$  is the length of the string

---

## OUTCOMES OF CHAPTER 4

- Understand and apply the concept of stack (LIFO) in programming.
- Perform basic operations: push, pop, peek, and check for overflow/underflow.
- Apply stacks in solving problems like expression evaluation and recursion.
- Implement stack using arrays and linked lists.
- Recognize stack use cases in compilers, parsing, and memory management.
- Debug and trace stack-based algorithms in practical scenarios.



## **SIMPLE LINEAR DATA STRUCTURE**





## SUB LESSON 5.1

### INTRODUCTION AND OPERATIONS ON QUEUE

#### OBJECTIVES OF CHAPTER 5

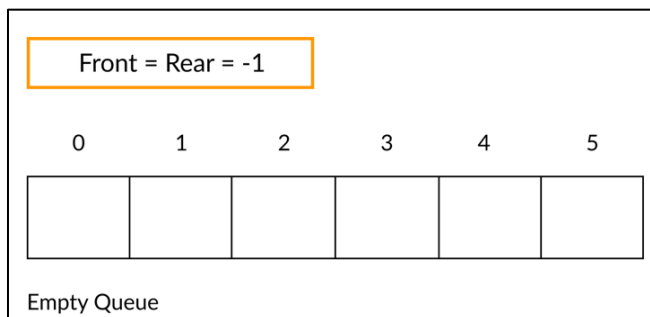
- Learn queue fundamentals and basic operations (FIFO).
- Understand different types of queues: circular, priority, and deque.
- Explore real-life applications like job scheduling and buffering.
- Implement queues using arrays and linked lists.
- Analyze advantages of various queue types.

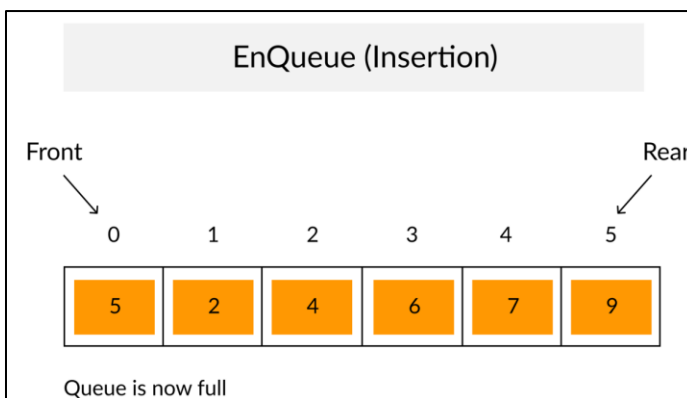
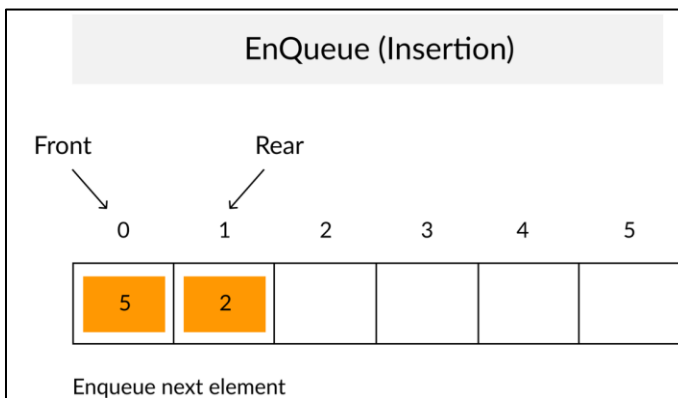
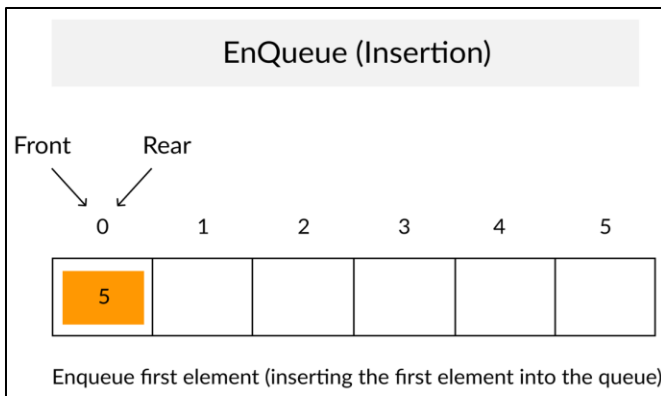
A queue is a linear data structure that follows the First-In-First-Out (FIFO) principle. It represents a collection of elements in which an element is added to one end called the rear and removed from the other end called the front. The element that has been in the queue the longest is at the front, and the newest element is at the rear. It operates like a queue of people waiting in line, where the person who arrives first is served first.

#### OPERATIONS ON QUEUE:

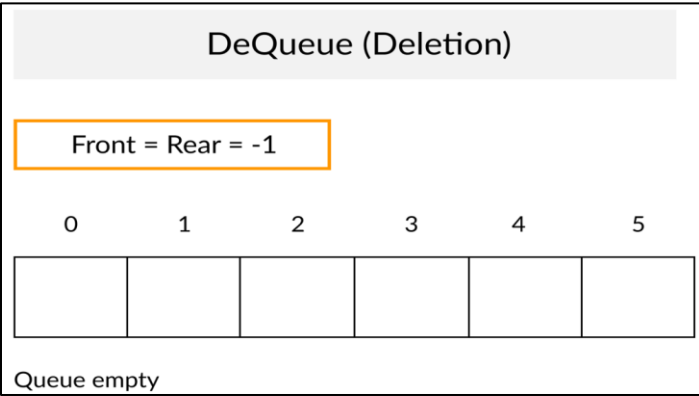
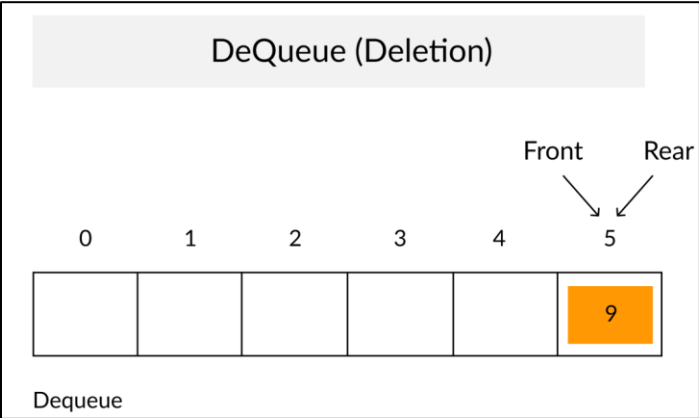
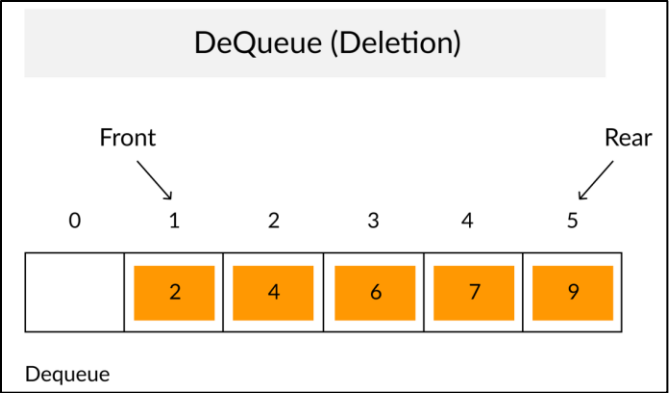
The fundamental operations performed on a queue are as follows:

1. **Enqueue:** This operation adds an element to the rear of the queue. It takes the element as a parameter and inserts it at the end of the queue.





2. Dequeue: This operation removes an element from the front of the queue. It returns the element that is removed, and the front is updated to the next element in the queue





---

## KEY TAKEAWAYS

- A queue follows the FIFO (First-In-First-Out) principle.
- Elements are added to the rear and removed from the front.
- The oldest element is at the front, while the newest element is at the rear.
- Enqueue: Adds an element to the rear of the queue.
- Dequeue: Removes an element from the front of the queue.





## **SIMPLE LINEAR DATA STRUCTURE**





## SUB LESSON 5.2

### TYPES OF QUEUE

#### TYPES OF QUEUE

There are different types of queues that are used in various scenarios based on specific requirements. Let's explore some of the commonly used types of queues:

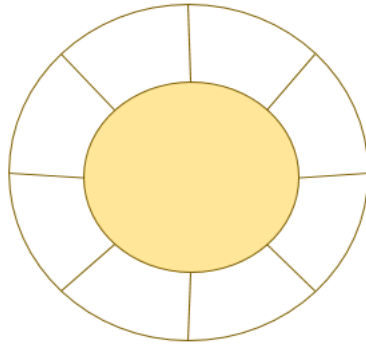
#### SIMPLE QUEUE:

- Simple Queue is the basic type of queue.
- It follows the FIFO (First-In-First-Out) principle.
- Elements are added to the rear and removed from the front.
- Only one end is used for enqueueing (rear) and dequeueing (front).



#### CIRCULAR QUEUE:

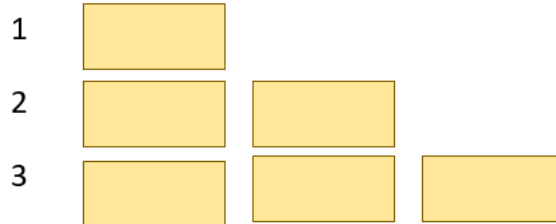
- Circular Queue is an extension of the simple queue.
- It overcomes the limitation of the simple queue where the rear and front may reach the end of the queue, making it impossible to enqueue new elements.
- In a circular queue, the last element is connected to the first element, creating a circular structure.
- This allows the rear and front to wrap around the queue and reuse the empty spaces.



### PRIORITY QUEUE:

- Priority Queue is a type of queue where each element has an associated priority value.
- Elements are dequeued based on their priority, not based on their arrival time.
- The element with the highest priority is dequeued first.
- Priority can be determined based on various factors, such as a numerical value, urgency level, or a custom-defined ordering criteria.

Priority



### DOUBLE ENDED QUEUE (DEQUE):

- Double Ended Queue, also known as Deque, is a queue in which elements can be added or removed from both ends.
- It allows insertion and removal of elements from both the front and rear.
- Deque can be used as both a queue and a stack, providing flexibility in implementing various algorithms.

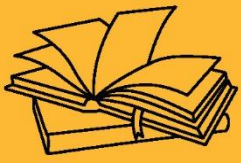


These are some of the commonly used types of queues. Each type has its own characteristics and is suitable for specific use cases and requirements. Choosing the right type of queue depends on the problem you are trying to solve and the behavior you need from the queue data structure.

---

### KEY TAKEAWAYS

- **Simple Queue:** Basic FIFO queue with elements added to the rear and removed from the front.
- **Circular Queue:** Overcomes the limitation of a simple queue by creating a circular structure, allowing reuse of empty spaces.
- **Priority Queue:** Elements have associated priorities, and dequeueing is based on priority rather than arrival time.
- **Double Ended Queue (Deque):** Supports insertion and removal of element
- **Simple Queue:** Basic FIFO queue with elements added to the rear and removed from the front.
- **Circular Queue:** Overcomes the limitation of a simple queue by creating a circular structure, allowing reuse of empty spaces.
- **Priority Queue:** Elements have associated priorities, and dequeueing is based on priority rather than arrival time.
- **Double Ended Queue (Deque):** Supports insertion and removal of element



## **SIMPLE LINEAR DATA STRUCTURE**





## SUB LESSON 5.3

### APPLICATIONS OF QUEUE

#### APPLICATIONS OF QUEUE

The queue data structure finds its applications in various domains and scenarios. Here are some common applications of the queue data structure:

**Job Scheduling:** Queues are used in job scheduling algorithms where tasks or jobs are queued and processed in the order they arrive. Examples include task scheduling in operating systems or job queues in batch processing systems.

**Breadth-First Search (BFS):** BFS is a graph traversal algorithm that explores all the vertices of a graph in breadth-first order. A queue is used to maintain the order of exploration by storing the vertices to be visited next.

**Printer Spooling:** When multiple users send print requests to a shared printer, a queue is used to manage the order in which the print jobs are processed. The jobs are added to the queue and printed in the order they were submitted.

**Process Synchronization:** In concurrent programming, queues can be used for process synchronization between multiple threads or processes.

**Message Queues:** Queues are widely used in message-oriented middleware and inter-process communication systems. They allow messages to be stored in a specific order and retrieved by consumers when they are ready to process them.

**CPU Scheduling:** In operating systems, queues are utilized in CPU scheduling algorithms. Different queues are used to prioritize and manage the execution of processes based on factors such as priority levels or time slices.



**Buffering:** Queues are commonly employed as buffers in data streaming applications. They help in managing the flow of data between different components or stages of a system to handle variations in data production and consumption rates.

**Event Handling:** In event-driven programming, queues can be used to manage events in the order they occur. Events are enqueued as they happen and processed sequentially, ensuring proper event handling and maintaining event order.

**Traffic Control :** In transportation systems, such as airport control systems or road networks, queues can be utilized to manage traffic flow.

These are just a few examples of how the queue data structure is applied in various domains. Queues are versatile and fundamental data structures that help in organizing and managing data in a first-in-first-out manner, making them useful in numerous real-world scenarios.

---

### KEY TAKEAWAYS

- The key takeaway of the basics of data structure is that it refers to the way data is organized and stored in a computer's memory or disk storage
- The primary goal of data structure is to provide an efficient way of accessing and manipulating data.
- Each data structure has its own strengths and weaknesses, and choosing the appropriate data structure depends on the specific use case and requirements.
- Understanding data structures is essential for computer programming and software development, as they provide a foundation for designing and implementing efficient algorithm
- By choosing the appropriate data structure and algorithm, programmers can optimize performance and improve the overall efficiency of their code.

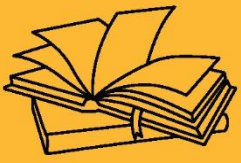


---

## OUTCOMES OF CHAPTER 5

- Describe the structure and working of queues (FIFO).
- Implement and manage various types of queues (simple, circular, priority).
- Apply queue data structures in real-world simulations and OS-level tasks.
- Compare queues with stacks and select based on problem needs.
- Solve practical problems using queue logic and operations.
- Analyze complexity and performance of queue-based solutions.





## COMPLEX LINEAR DATA STRUCTURE

---





## SUB LESSON 6.1

### INTRODUCTION AND TYPES OF LINKED LIST

#### OBJECTIVES OF CHAPTER 6

- Understand structure and types: singly, doubly, and circular linked lists.
- Perform insertion, deletion, and traversal operations.
- Compare linked lists with arrays for memory efficiency.
- Study applications in memory management and dynamic data handling.
- Solve problems involving various linked list operations..

A linked list is a data structure that consists of a sequence of elements called nodes, where each node contains both data and a reference (or link) to the next node in the sequence. Unlike arrays, linked lists do not require contiguous memory allocation. Instead, they utilize dynamic memory allocation, allowing efficient insertion and deletion of elements at any position.

#### TYPES OF LINKED LIST:

There are several types of linked lists, each with its own characteristics and variations. Let's explore some common types:

#### SINGLY LINKED LIST:

In a singly linked list, each node contains data and a reference to the next node in the sequence.

It forms a unidirectional chain of nodes, where the last node points to null, indicating the end of the list.

Traversing a singly linked list is done sequentially from the first node (head) to the last node.



A node can be represented as,



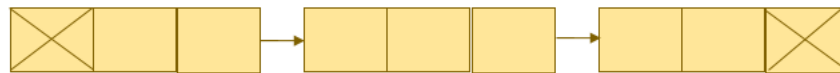
```
struct node  
{  
    int data;  
    struct node *next;  
}
```

### DOUBLY LINKED LIST:

A doubly linked list extends the singly linked list by including an additional reference to the previous node in each node.

Each node has references to both the next and previous nodes, forming a bidirectional chain.

This allows traversal in both forward and backward directions, enhancing flexibility but requiring more memory for the extra references.



A node can be represented as,

```
struct node  
{  
    int data;  
    struct node *next;  
    struct node *prev;  
}
```

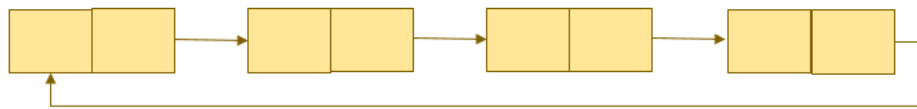


## CIRCULAR LINKED LIST:

In a circular linked list, the last node of the list points back to the first node, forming a circular structure.

This enables continuous traversal from any node, as the last node loops back to the first node.

Circular linked lists are useful in scenarios where circular behavior is desired or when there is a need for efficient rotation or cycling of elements.



A node can be represented as,

struct node

```
{  
  
    int data;  
  
    struct node *next;  
  
}
```



---

## KEY TAKEAWAYS

- Linked lists are data structures consisting of nodes with data and references to the next node.
- They do not require contiguous memory allocation and allow efficient insertion and deletion operations.
- Each node in a singly linked list contains data and a reference to the next node.
- Traversal is done sequentially from the head to the last node.
- Singly linked lists are simple and memory-efficient.
- Doubly linked lists have nodes with references to both the next and previous nodes.
- They allow traversal in both forward and backward directions.
- Doubly linked lists provide flexibility at the cost of slightly more memory usage.
- In circular linked lists, the last node points back to the first node, forming a circular structure.
- Continuous traversal is possible from any node in the circular list.
- Circular linked lists are useful for circular behavior and efficient rotation or cycling of elements.
- Understanding the different types of linked lists helps in selecting the appropriate linked list implementation based on specific requirements. Each type offers different features and trade-offs in terms of traversal, memory usage, and functionality.



## COMPLEX LINEAR DATA STRUCTURE

---



## SUB LESSON 6.2

### OPERATION ON SINGLY LINKED LIST

A singly linked list is a linear data structure where each element (node) consists of two parts: data and a pointer (or reference) to the next node in the sequence. The last node's pointer typically points to NULL, signifying the end of the list.

#### STRUCTURE OF A SINGLY LINKED LIST NODE

```
struct Node {  
  
    int data;    // Data stored in the node  
  
    struct Node* next; // Pointer to the next node  
  
};
```

#### I. TRAVERSAL OPERATION

Traversal involves visiting each node in the linked list, typically from the head to the tail, to perform some action (e.g., print data, search for an element, count nodes).

##### **Steps:**

Start with a pointer, say current, initialized to the head of the list.

Loop while current is not NULL (meaning you haven't reached the end of the list).

##### **Inside the loop:**

Access or process the data of the current node.

Move current to the next node by setting `current = current->next`.

Time Complexity:  $O(N)$  - Linear time, as it requires visiting each of the  $N$  nodes in the worst case.

Space Complexity:  $O(1)$  - Constant space, as it only uses a few pointers regardless of the list size.



Example (Conceptual):

List: head -> A -> B -> C -> NULL

**Traversal Steps:**

current points to A. Print A.

current moves to B. Print B.

current moves to C. Print C.

current moves to NULL. Loop terminates.

**Common Uses of Traversal:**

Printing the list elements: Displaying all data values.

Searching for an element: Finding if a specific value exists in the list.

Counting nodes: Determining the total number of elements.

Finding the last node (for insertion at end): As discussed in insertion at the end.

Finding the node before a target (for insertion/deletion at position): As discussed in insertion and deletion at a desired position.

---

## II. INSERTION OPERATIONS

Inserting a new node into a singly linked list involves creating the node and then adjusting pointers to incorporate it into the existing sequence.

### 1. INSERTION AT THE BEGINNING (HEAD)

This is the simplest insertion. The new node becomes the new head of the list.

**Steps:**

Create a new node.





Assign data to the new node.

Make the next pointer of the new node point to the current head of the list.

Update the head of the list to point to the new node.

Time Complexity:  $O(1)$  - Constant time, as it only involves a few pointer manipulations regardless of list size.

Example (Conceptual):

Before: head  $\rightarrow$  A  $\rightarrow$  B  $\rightarrow$  C

Insert X:

new\_node (X)

new\_node  $\rightarrow$  next = head (i.e., new\_node  $\rightarrow$  next points to A)

head = new\_node (i.e., head now points to X)

After: head  $\rightarrow$  X  $\rightarrow$  A  $\rightarrow$  B  $\rightarrow$  C

## 2. INSERTION AT THE END (TAIL)

Inserting a new node at the end requires traversing the list to find the last node.

### **Steps:**

Create a new node.

Assign data to the new node.

Make the next pointer of the new node point to NULL (as it will be the new last node).

If the list is empty, the new node becomes the head.

If the list is not empty, traverse the list until you reach the last node (the one whose next pointer is NULL).



Make the next pointer of the last node point to the new node.

Time Complexity:  $O(N)$  - Linear time, as it requires traversing the entire list in the worst case (when the list is very long).

Example (Conceptual):

Before: head  $\rightarrow$  A  $\rightarrow$  B  $\rightarrow$  C

Insert X:

new\_node (X)

new\_node  $\rightarrow$  next = NULL

Traverse to C.

C  $\rightarrow$  next = new\_node

After: head  $\rightarrow$  A  $\rightarrow$  B  $\rightarrow$  C  $\rightarrow$  X

### 3. INSERTION AT A DESIRED POSITION (MIDDLE/SPECIFIC INDEX)

Inserting a node at a specific position (e.g., after a particular node, or at a given index) requires traversing to the node before the desired insertion point.

#### **Steps:**

Create a new node.

Assign data to the new node.

Handle edge cases:

If the position is 1 (beginning), it's the same as "Insertion at the Beginning".

If the list is empty and the position is not 1, it's an invalid operation.



Traverse the list to the node before the desired insertion point. Keep a current pointer and a count (or index).

Make the next pointer of the new node point to the node that `current->next` was pointing to.

Make `current->next` point to the new node.

Handle cases where the desired position is out of bounds (e.g., beyond the end of the list).

Time Complexity:  $O(N)$  - Linear time, as it may require traversing a significant portion of the list.

Example (Conceptual):

Before: head -> A -> B -> C

Insert X after B:

`new_node(X)`

Traverse to B (let this be current).

`new_node->next = current->next` (i.e., `new_node->next` points to C)

`current->next = new_node` (i.e., B->next points to X)

After: head -> A -> B -> X -> C

---

## II. DELETION OPERATIONS

Deleting a node from a singly linked list involves finding the node to be deleted and then adjusting pointers to bypass it. The memory occupied by the deleted node should also be freed to prevent memory leaks.

### 1. DELETION FROM THE BEGINNING (HEAD)

This is the simplest deletion. The head of the list is removed.

**Steps:**

Check if the list is empty. If so, return.

Store a temporary pointer to the current head.

Update the head of the list to point to the next node.

Free the memory of the temporary node.

Time Complexity:  $O(1)$  - Constant time.

Example (Conceptual):

Before: head  $\rightarrow$  A  $\rightarrow$  B  $\rightarrow$  C

Delete A:

temp = head (i.e., temp points to A)

head = head  $\rightarrow$  next (i.e., head points to B)

free(temp)

After: head  $\rightarrow$  B  $\rightarrow$  C

## 2. DELETION FROM THE END (TAIL)

Deleting the last node requires traversing the list to find the second-to-last node.

**Steps:**

Check if the list is empty. If so, return.

If there's only one node, delete it and set head to NULL.

Traverse the list using two pointers: current (to the last node) and previous (to the second-to-last node).

Make previous  $\rightarrow$  next point to NULL.



Free the memory of the current node.

Time Complexity:  $O(N)$  - Linear time, as it requires traversing the entire list.

Example (Conceptual):

Before: head  $\rightarrow$  A  $\rightarrow$  B  $\rightarrow$  C

Delete C:

Traverse with previous at B and current at C.

previous $\rightarrow$ next = NULL (i.e., B $\rightarrow$ next points to NULL)

free(current) (i.e., free(C))

After: head  $\rightarrow$  A  $\rightarrow$  B

### 3. DELETION FROM A DESIRED POSITION (MIDDLE/SPECIFIC VALUE)

Deleting a node at a specific position (e.g., a node with a particular data value, or at a given index) requires finding the node before the one to be deleted.

#### **Steps:**

Check if the list is empty.

If the head node contains the data to be deleted, perform "Deletion from the Beginning".

Traverse the list with two pointers: current and previous. current searches for the node to be deleted, and previous trails one node behind current.

Once the node to be deleted (current) is found:

Make previous $\rightarrow$ next point to current $\rightarrow$ next.

Free the memory of the current node.

Handle cases where the data is not found in the list.



Example (Conceptual - Delete B by value):

Before: head -> A -> B -> C

Delete B:

Traverse with previous at A and current at B.

previous->next = current->next (i.e., A->next points to C)

free(current) (i.e., free(B))

After: head -> A -> C

| Operation | Variation           | Time Complexity | Description                                                               |
|-----------|---------------------|-----------------|---------------------------------------------------------------------------|
| Traversal | N/A                 | O(N)            | Visit each node from head to tail, typically for processing or searching. |
| Insertion | At Beginning        | O(1)            | New node becomes the head; simple pointer updates.                        |
|           | At End              | O(N)            | Traverse to the last node; append new node.                               |
|           | At Desired Position | O(N)            | Traverse to the node before the insertion point; adjust pointers.         |
| Deletion  | From Beginning      | O(1)            | Head node is removed; simple pointer updates.                             |



|  |                       |        |                                                                          |
|--|-----------------------|--------|--------------------------------------------------------------------------|
|  | From End              | $O(N)$ | Traverse to the second-to-last node; remove the last node.               |
|  | From Desired Position | $O(N)$ | Traverse to the node before the target node; bypass and free the target. |

### KEY TAKEAWAYS

- Nodes and Pointers are Core: Singly linked lists fundamentally rely on nodes (data + next pointer) and pointer manipulation for all operations.
- Head is Crucial: The head pointer is the list's entry point, essential for accessing and modifying the list, especially at the beginning.
- Traversal is Fundamental: Most operations beyond the head require  $O(N)$  traversal through the list to locate the desired position.
- Time Complexity Varies: Operations at the beginning are  $O(1)$ , while those at the end or a specific middle position are  $O(N)$  due to traversal.
- Memory Management is Key: Proper memory deallocation (e.g., using `free()`) is vital during deletion to prevent memory leaks.
- Edge Cases Matter: Robust code must explicitly handle edge cases like empty lists, single-node lists, or invalid positions to avoid errors.



## COMPLEX LINEAR DATA STRUCTURE







## SUB LESSON 6.3

### OPERATION ON DOUBLY LINKED LIST

A doubly linked list is a linear data structure where each element (node) consists of three parts: data, a pointer to the next node (next), and a pointer to the previous node (prev). This bidirectional linking allows for traversal in both forward and backward directions, offering more flexibility compared to singly linked lists.

The head pointer points to the first node, and a tail (or last) pointer often points to the last node. The prev pointer of the first node and the next pointer of the last node typically point to NULL.

#### STRUCTURE OF A DOUBLY LINKED LIST NODE.

```
struct Node {  
  
    int data;      // Data stored in the node  
  
    struct Node* prev; // Pointer to the previous node  
  
    struct Node* next; // Pointer to the next node  
  
};
```

#### I. TRAVERSAL OPERATIONS

Doubly linked lists offer the advantage of bidirectional traversal.

##### 1. FORWARD TRAVERSAL

###### **Steps:**

Start with a pointer, say current, initialized to the head of the list.

Loop while current is not NULL.

Process current->data.



Move current = current->next.

Time Complexity:  $O(N)$

## 2. BACKWARD TRAVERSAL

### **Steps:**

Start with a pointer, say current, initialized to the tail (or last) of the list. If tail isn't maintained, you'd first traverse to find the last node.

Loop while current is not NULL.

Process current->data.

Move current = current->prev.

Time Complexity:  $O(N)$  (or  $O(1)$  if tail is directly accessible)

---

## II. INSERTION OPERATIONS

Inserting a new node involves creating the node and adjusting up to four pointers (two next and two prev) to maintain the bidirectional links.

### 1. INSERTION AT THE BEGINNING (HEAD)

#### **Steps:**

Create a new node, assign data, set new\_node->prev = NULL.

Set new\_node->next = head.

If the list is not empty, set head->prev = new\_node.

Update head = new\_node.

If the list was empty, new\_node also becomes the tail.

Time Complexity:  $O(1)$



Example (Conceptual):

Before:  $\text{NULL} \leftrightarrow A \leftrightarrow B \leftrightarrow C \rightarrow \text{NULL}$  (head points to A)

Insert X:

$\text{new\_node}(X)$

$\text{new\_node} \rightarrow \text{prev} = \text{NULL}$

$\text{new\_node} \rightarrow \text{next} = A$

$A \rightarrow \text{prev} = \text{new\_node}$

$\text{head} = \text{new\_node}$

After:  $\text{NULL} \leftrightarrow X \leftrightarrow A \leftrightarrow B \leftrightarrow C \rightarrow \text{NULL}$

## 2. INSERTION AT THE END (TAIL)

### **Steps:**

Create a new node, assign data, set  $\text{new\_node} \rightarrow \text{next} = \text{NULL}$ .

If the list is empty,  $\text{new\_node}$  becomes both head and tail, and  $\text{new\_node} \rightarrow \text{prev} = \text{NULL}$ .

If the list is not empty:

Set  $\text{new\_node} \rightarrow \text{prev} = \text{tail}$ .

Set  $\text{tail} \rightarrow \text{next} = \text{new\_node}$ .

Update  $\text{tail} = \text{new\_node}$ .

Time Complexity:  $O(1)$  if tail pointer is maintained;  $O(N)$  if you need to traverse to find the last node. (Typically, tail is maintained in a DLL for  $O(1)$  end operations).

Example (Conceptual):

Before:  $\text{NULL} \leftrightarrow A \leftrightarrow B \leftrightarrow C \rightarrow \text{NULL}$  (tail points to C)



Insert X:

```
new_node (X)
```

```
new_node->next = NULL
```

```
new_node->prev = C
```

```
C->next = new_node
```

```
tail = new_node
```

After: NULL <- A <-> B <-> C <-> X -> NULL

### 3. INSERTION AT A DESIRED POSITION (MIDDLE/AFTER A SPECIFIC NODE)

Steps (Insert after a given\_node):

Create a new node, assign data.

Handle edge cases: If given\_node is NULL or not found.

Set new\_node->next = given\_node->next.

Set new\_node->prev = given\_node.

If given\_node->next is not NULL (i.e., not inserting at the very end):

Set given\_node->next->prev = new\_node.

Set given\_node->next = new\_node.

If new\_node->next is NULL (i.e., new\_node is now the new tail), update tail = new\_node.

Time Complexity: O(N) if you need to search for given\_node; O(1) if given\_node reference is provided.

Example (Conceptual - Insert X after B):

Before: NULL <- A <-> B <-> C -> NULL



Insert X:

```
new_node (X)
```

```
new_node->next = C
```

```
new_node->prev = B
```

```
C->prev = new_node
```

```
B->next = new_node
```

After: NULL <- A <-> B <-> X <-> C -> NULL

---

### III. DELETION OPERATIONS

Deleting a node involves adjusting pointers of its neighbors and freeing the node's memory.

#### 1. DELETION FROM THE BEGINNING (HEAD)

**Steps:**

Check if the list is empty. If so, return.

Store head in a temporary pointer temp.

Update head = head->next.

If the new head is not NULL, set head->prev = NULL.

If temp was also the tail (meaning only one node existed), set tail = NULL.

Free temp.

Time Complexity:  $O(1)$

Example (Conceptual):

Before: NULL <- A <-> B <-> C -> NULL



Delete A:

temp = A

head = B

B->prev = NULL

free(temp)

After: NULL <- B <-> C -> NULL

## 2. DELETION FROM THE END (TAIL)

### **Steps:**

Check if the list is empty. If so, return.

If there's only one node (head == tail), set head = NULL, tail = NULL, then free the node.

Store tail in a temporary pointer temp.

Update tail = tail->prev.

Set tail->next = NULL.

Free temp.

Time Complexity:  $O(1)$  if tail pointer is maintained;  $O(N)$  if you need to traverse to find the second-to-last node. (Typically, tail is maintained for  $O(1)$  end operations).

Example (Conceptual):

Before: NULL <- A <-> B <-> C -> NULL

Delete C:

temp = C



tail = B

B->next = NULL

free(temp)

After: NULL <- A <-> B -> NULL

### 3. DELETION AT A DESIRED POSITION (MIDDLE/SPECIFIC VALUE)

*Steps (Delete a target\_node):*

Check if the list is empty or target\_node is NULL.

If target\_node is the head, perform "Deletion from the Beginning".

If target\_node is the tail, perform "Deletion from the End".

If target\_node is in the middle:

Set target\_node->prev->next = target\_node->next.

Set target\_node->next->prev = target\_node->prev.

Free target\_node.

Time Complexity: O(N) if you need to search for target\_node; O(1) if target\_node reference is provided.

Example (Conceptual - Delete B):

Before: NULL <- A <-> B <-> C -> NULL

Delete B:

A->next = C

C->prev = A

free(B)



After: NULL <- A <-> C -> NULL

| Operation | Variation             | Time Complexity | Description                                                                                                           |
|-----------|-----------------------|-----------------|-----------------------------------------------------------------------------------------------------------------------|
| Traversal | Forward               | O(N)            | Visit each node from head to tail using next pointers.                                                                |
|           | Backward              | O(N)            | Visit each node from tail to head using prev pointers.                                                                |
| Insertion | At Beginning          | O(1)            | New node becomes head; update head and its new next's prev.                                                           |
|           | At End                | O(1)            | New node becomes tail; update tail and its new prev's next. (Assumes tail pointer is maintained)                      |
|           | At Desired Position   | O(N) or O(1)    | Adjust next and prev pointers of new node and its neighbors. O(N) if search is needed, O(1) otherwise.                |
| Deletion  | From Beginning        | O(1)            | Remove head; update new head's prev to NULL.                                                                          |
|           | From End              | O(1)            | Remove tail; update new tail's next to NULL. (Assumes tail pointer is maintained)                                     |
|           | From Desired Position | O(N) or sO(1)   | Bypass target node by linking its neighbors; update next and prev pointers. O(N) if search is needed, O(1) otherwise. |

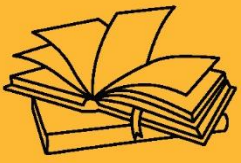




---

## KEY TAKEAWAYS

- **Bidirectional Linking:** Doubly linked lists gain significant flexibility from their prev pointers, allowing efficient (often  $O(1)$ ) traversal and manipulation in both forward and backward directions.
- **Efficient End Operations:** Unlike singly linked lists, insertion and deletion at the end are typically  $O(1)$  in doubly linked lists because a tail pointer provides direct access to the last node.
- **Increased Memory & Complexity:** The added prev pointer in each node means higher memory consumption per node and slightly more complex pointer manipulations for each operation compared to singly linked lists.
- **$O(1)$  Middle Operations (with reference):** If you have a direct reference to the node you want to insert next to or delete, operations in the middle of the list become  $O(1)$ , bypassing the need for traversal.
- **Traversal Still  $O(N)$ :** While individual operations might be faster, traversing the entire list (forward or backward) still requires visiting every node, thus remaining an  $O(N)$  operation.
- **Robust Pointer Management:** Due to four potential pointer updates per operation (two next, two prev), meticulous attention to pointer adjustments and null checks is crucial for correct implementation.



## COMPLEX LINEAR DATA STRUCTURE





## SUB LESSON 6.4

### APPLICATION OF LINKED LIST

#### APPLICATIONS OF LINKED LIST

Linked lists have a wide range of applications across various domains. Here are some common applications of linked lists:

**Implementation of Data Structures:** Linked lists serve as a fundamental building block for implementing other complex data structures such as stacks, queues, hash tables, graphs, and trees. The dynamic nature of linked lists enables efficient insertion, deletion, and manipulation of elements.

**Dynamic Memory Allocation:** Linked lists are used in memory management systems to allocate and deallocate memory dynamically. They allow efficient utilization of memory by allocating memory blocks of varying sizes as needed.

**Text Editors:** Linked lists are utilized in text editors to represent and manipulate the contents of a document. Each line of text can be stored as a node in a linked list, facilitating operations such as insertion, deletion, and traversal of text.

**Music and Video Playlists:** Linked lists are often used to implement playlists in music and video players. Each song or video is represented by a node, and the links connect them in the desired order. Users can navigate through the playlist by traversing the linked list.

**Operating System Task Management:** Operating systems employ linked lists for task management, scheduling, and process control. Linked lists can represent queues of processes, ready queues, waiting queues, and various other task-related data structures.

**Sparse Matrix Representation:** Sparse matrices, which have a large number of zero elements, can be efficiently represented using linked lists. Each non-zero element is represented by a node with the value, row index, and column index, reducing memory usage.



**Symbol Tables:** Linked lists are used in symbol tables or hash tables to handle collisions. When multiple values hash to the same index, they can be stored in a linked list at that index, allowing efficient retrieval and management of keys and associated values.

**Undo/Redo Functionality:** In applications that require undo/redo functionality, linked lists can be used to store a history of actions. Each action is stored as a node, and traversing the linked list allows users to undo or redo operations.

**Navigation Systems:** Linked lists are employed in navigation systems to represent and traverse routes or paths. Each node represents a point of interest or location, and the links define the sequence of locations to follow.

These are just a few examples of how linked lists are applied in different domains. The flexibility and dynamic nature of linked lists make them suitable for various scenarios that require efficient data organization and manipulation.

---

## KEY TAKEAWAYS

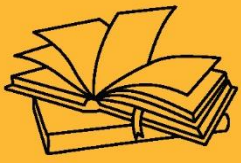
- Linked lists have a wide range of applications across different domains.
- They are used to implement other data structures like stacks, queues, hash tables, graphs, and trees.
- Linked lists are useful for dynamic memory allocation, allowing efficient use of memory by allocating and deallocating blocks as needed.
- They are employed in text editors, music and video playlists, operating system task management, and sparse matrix representation.
- Linked lists are utilized in symbol tables or hash tables to handle collisions and in undo/redo functionality in applications.
- Navigation systems make use of linked lists to represent and traverse routes or paths.
- Linked lists offer flexibility, efficient insertion and deletion, and dynamic memory allocation, making them suitable for various scenarios.
- Understanding the applications of linked lists helps in recognizing their versatility and selecting them as an appropriate data structure for specific use cases



---

## OUTCOMES OF CHAPTER 6

- Understand and explain the concept and types of linked lists.
- Perform insertion, deletion, and traversal on singly and doubly linked lists.
- Implement linked lists to manage dynamic memory efficiently.
- Identify appropriate use cases for linked lists over arrays.
- Write and test programs for different linked list operations.
- Apply linked lists in real-life applications like memory management and navigation systems.



## **NON-LINEAR DATA STRUCTURE**

---



## SUB LESSON 7.1

### BASIC TERMINOLOGIES OF TREE

#### OBJECTIVES OF CHAPTER 7

- Learn basic tree terminology and representation.
- Explore types: binary, BST, AVL, and B-trees.
- Perform operations like insertion, deletion, and traversal on trees.
- Understand tree balancing techniques.
- Apply trees in hierarchical data storage and searching..

#### BASIC TERMINOLOGIES OF TREE

A tree data structure is a widely used and versatile data structure in computer science. It is a hierarchical data structure that represents a set of elements called nodes. These nodes are connected by edges to form a tree-like structure.

At the core of a tree structure is the root node, which serves as the starting point of the tree. Each node in the tree can have zero or more child nodes, except for the leaf nodes that have no children. Nodes that have the same parent node are called siblings.

Some of the basic terminologies of tree data structure is as follows:

**Node:** A node is a fundamental unit of a tree data structure. It represents a single element or data point in the tree.

**Root:** The root is the topmost node of the tree. It is the starting point or the parent node of all other nodes in the tree.

**Parent:** A parent node is a node that has one or more child nodes. It is located higher in the hierarchy compared to its child nodes.



**Child:** A child node is a node that has a parent node. It is located below its parent node in the tree structure.

**Siblings:** Sibling nodes are nodes that share the same parent node. In other words, siblings are nodes that are at the same level in the tree and have the same parent.

**Leaf:** A leaf node, also known as a terminal node, is a node that does not have any child nodes. It is located at the end of a branch in the tree.

**Edge:** An edge is a connection or link between two nodes in a tree. It represents the relationship between a parent and its child node.

**Depth:** The depth of a node refers to the number of edges from the root to that node. The root node has a depth of 0.

**Height:** The height of a node is the number of edges on the longest path from that node to a leaf node. The height of the tree is the height of the root node.

**Subtree:** A subtree is a portion of a tree that consists of a node (called the subtree's root) and all its descendants. It forms a smaller tree within the larger tree.

**Binary Tree:** A binary tree is a type of tree in which each node can have at most two children, referred to as the left child and the right child.

**Binary Search Tree (BST):** A binary search tree is a binary tree in which the left child of a node contains a value less than or equal to the node, and the right child contains a value greater than the node. It allows for efficient searching, insertion, and deletion operations.

These are some of the basic terminologies associated with tree data structures. There are more advanced concepts and variations depending on the specific type of tree data structure.

A

/ | \

B C D





/ | \

E F G

In this example, we have a tree with six nodes labeled A, B, C, D, E, F, and G. The root node is A, and it has three child nodes: B, C, and D.

Node B has one child node, E, while node D has two child nodes, F and G. Node C has no child nodes and is a leaf node. Nodes E, F, and G are also leaf nodes since they have no child nodes.

The connections between nodes are represented by edges. For example, there is an edge between node A and its child nodes B, C, and D. Similarly, there are edges between nodes B and E, D and F/G, and so on.

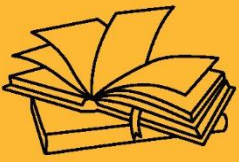
This is just a simple example to illustrate the structure of a tree. In real-world scenarios, trees can be much larger and more complex, with multiple levels and branches.



---

## KEY TAKEAWAYS

- **Node:** A fundamental unit of a tree that holds a value or data.
- **Root:** The topmost node in a tree structure, serving as the starting point. It has no parent.
- **Parent:** A node that has one or more child nodes directly connected below it.
- **Child:** A node directly connected to and descending from a parent node.
- **Sibling:** Nodes that share the same parent node.
- **Leaf:** Also known as a terminal node or external node, it is a node with no child nodes.
- **Internal node:** A node that has at least one child node.
- **Edge:** The connection or link between two nodes in a tree.
- **Depth:** The depth of a node is the number of edges from the root to that node.
- **Height:** The height of a node is the number of edges on the longest path from that node to a leaf node.
- **Subtree:** A portion of a tree consisting of a node and its descendants, forming a smaller tree.
- **Binary Tree:** A type of tree in which each node has at most two child nodes, referred to as the left child and the right child.
- **Binary Search Tree (BST):** A binary tree where the left child of a node contains a value less than or equal to the node, and the right child contains a value greater than the node.



## **NON-LINEAR DATA STRUCTURE**

---



## SUB LESSON 7.2

### TYPES OF TREE

#### TYPES OF TREE

There are several types of trees used in computer science and data structures. Here are a few common types with examples:

**Binary Tree:** A binary tree is a tree data structure in which each node has at most two child nodes, referred to as the left child and the right child. It is one of the simplest and most commonly used tree structures.

A

/ \

B C

/\ /

D E F

**Binary Search Tree (BST):** A binary search tree is a binary tree where the left child of a node contains a value less than or equal to the node, and the right child contains a value greater than the node. It enables efficient searching, insertion, and deletion operations.

5

/ \

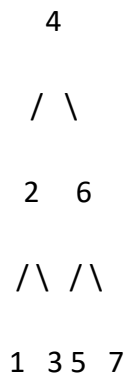
3 7

/\ /\

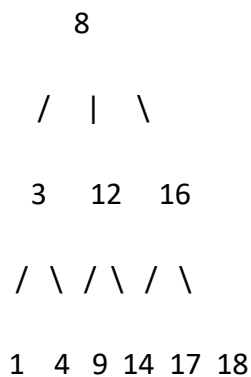
2 4 6 8



**AVL Tree:** An AVL tree is a self-balancing binary search tree. It maintains a balance factor for each node, ensuring that the heights of the left and right subtrees differ by at most one. This balance allows for efficient operations.



**B-Tree:** A B-tree is a self-balancing tree that can have more than two children per node. It is commonly used in databases and file systems, as it allows for efficient disk access and retrieval.

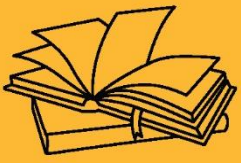




---

## KEY TAKEAWAYS

- Trees are hierarchical data structures that consist of nodes connected by edges.
- The root node is the topmost node in a tree, and it serves as the starting point.
- Binary trees allow each node to have at most two child nodes, typically referred to as the left child and the right child.
- Binary search trees (BSTs) are binary trees that have an order property, with the left child containing smaller values and the right child containing larger values.
- AVL tree is self-balancing binary search tree that ensure the height of the tree remains balanced, resulting in efficient operations.
- B-trees are self-balancing trees that allow for more than two child nodes per node. They are commonly used in databases and file systems.
- Understanding different types of trees and their properties helps in selecting the appropriate tree structure based on the requirements of a particular problem or application.



## NON-LINEAR DATA STRUCTURE





## SUB LESSON 7.3

### OPERATION ON BINARY SEARCH TREE

#### OPERATION ON BINARY SEARCH TREE

Binary search trees (BSTs) are a type of binary tree data structure that satisfies the binary search property. The binary search property states that for any node in the tree, all the nodes in its left subtree have values smaller than the node's value, and all the nodes in its right subtree have values greater than the node's value. BSTs allow efficient insertion, deletion, and searching operations. Let's explore these operations with examples:

#### INSERTION:

To insert a value into a BST, we start at the root node and compare the value with the current node's value.

If the value is smaller, we go to the left subtree. If the value is larger, we go to the right subtree.

We continue traversing the tree until we find an empty spot (a leaf node or null pointer) where the new node can be inserted.

Here's an example of inserting the values 5, 3, 7, 1, and 4 into an empty BST:

```
      5
     /\
    3  7
   /\
  1  4
```

#### SEARCHING:





To search for a value in a BST, we start at the root node and compare the value with the current node's value.

If the value matches, we have found the node.

If the value is smaller, we go to the left subtree. If the value is larger, we go to the right subtree.

We repeat this process until we find the node with the matching value or reach a null pointer (indicating the value is not present in the BST).

Here's an example of searching for the value 4 in the BST from the previous example. The search path is shown in bold:

```
    5
   /\
  3  7
 /\
1  4
```

Search path: 5 → 3 → 4 (found)

#### DELETION:

Deleting a node from a BST requires considering three cases:

- a) If the node to be deleted is a leaf node (has no children), we can simply remove it.
- b) If the node to be deleted has one child, we can replace the node with its child.
- c) If the node to be deleted has two children, we need to find either the maximum value in its left subtree or the minimum value in its right subtree to replace the node's value. This ensures that the binary search property is maintained.

After finding the replacement node, we recursively delete it from the left or right subtree.

Here's an example of deleting the value 3 from the BST:

```
5
 / \
3  7
 / \
1  4
```

After Deletion :

```
5
 / \
4  7
 /
1
```

#### TRAVERSAL:

Consider the following binary tree:

```
4
 / \
2  6
 / \ \
1  3 5 7
```



### INORDER TRAVERSAL:

- In inorder traversal, we visit the nodes of the binary tree in the order of left subtree, root, and then right subtree.
- Here are the steps involved in inorder traversal:
- Traverse the left subtree recursively by calling the inorder traversal function on the left child.
- Visit the current node (root) and process it.
- Traverse the right subtree recursively by calling the inorder traversal function on the right child.
- Using inorder traversal on the given binary tree, the order of visited nodes will be: 1, 2, 3, 4, 5, 6, 7.

### PREORDER TRAVERSAL:

- In preorder traversal, we visit the nodes of the binary tree in the order of root, left subtree, and then right subtree.
- Here are the steps involved in preorder traversal:
- Visit the current node (root) and process it.
- Traverse the left subtree recursively by calling the preorder traversal function on the left child.
- Traverse the right subtree recursively by calling the preorder traversal function on the right child.
- Using preorder traversal on the given binary tree, the order of visited nodes will be: 4, 2, 1, 3, 6, 5, 7.

### POSTORDER TRAVERSAL:

- In postorder traversal, we visit the nodes of the binary tree in the order of left subtree, right subtree, and then the root.
- Here are the steps involved in postorder traversal:



- Traverse the left subtree recursively by calling the postorder traversal function on the left child.
- Traverse the right subtree recursively by calling the postorder traversal function on the right child.
- Visit the current node (root) and process it.
- Using postorder traversal on the given binary tree, the order of visited nodes will be: 1, 3, 2, 5, 7, 6, 4.

By following these steps and understanding the order of visiting nodes, one can perform inorder, preorder, and postorder traversals on binary trees. These traversals are useful for various applications, such as evaluating arithmetic expressions, creating expression trees, and extracting the hierarchical structure of a tree.

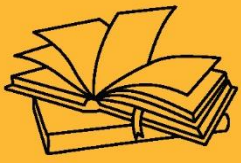
These are the basic operations on binary search trees. It's important to note that maintaining the binary search property is crucial to ensure the efficiency of these operations



---

## KEY TAKEAWAYS

- **Binary Search Property:** BSTs maintain the binary search property, which allows for efficient searching, insertion, and deletion operations. The property ensures that all values in the left subtree of a node are smaller, and all values in the right subtree are greater than the node's value.
- **Insertion:** When inserting a new value into a BST, we compare the value with each node and traverse the tree to find the appropriate position for insertion. Smaller values go to the left subtree, and larger values go to the right subtree. The insertion process continues until an empty spot is found, where the new node is inserted.
- **Searching:** Searching for a value in a BST involves comparing the value with each node and traversing left or right based on the comparison. The search process continues until the matching value is found or a null pointer is encountered, indicating the value is not present in the BST.
- **Deletion:** Deleting a node from a BST requires considering three cases: a) If the node has no children (leaf node), it can be directly removed. b) If the node has one child, it can be replaced by its child. c) If the node has two children, a replacement node from either the left subtree's maximum value or the right subtree's minimum value is found to preserve the binary search property. After finding the replacement, it is recursively deleted from its original position.
- **Traversal:** Traversal can be of three types, depends on the application one can choose inorder, preorder and post order traversing by following their rules.
- **Maintenance of Binary Search Property:** To ensure the efficiency and correctness of operations, it's essential to maintain the binary search property during BST operations. This property allows for efficient searching and preserves the ordered structure of the BST.



## NON-LINEAR DATA STRUCTURE

---





## SUB LESSON 7.4

### OPERATION ON AVL TREE

#### OPERATION ON AVL TREE

AVL trees are a type of self-balancing binary search tree (BST) that maintain a balanced structure to ensure efficient operations. The balancing is achieved by enforcing a height balance property, where the heights of the left and right subtrees of any node differ by at most one. AVL trees perform the same operations as regular BSTs, but with additional steps to maintain the balance. Let's explore the operations on AVL trees with examples:

Before diving into the AVL tree operation, one must know that after performing any operation, AVL tree must be balanced and that can be achieved by performing different rotation operations like left-left, left-right, right-right, and right-left.

#### LEFT-LEFT ROTATION:

This rotation is used when the left subtree of the left child of a node becomes taller by two levels compared to the right subtree.

To restore balance, a left-left rotation is performed by rotating the unbalanced node and its left child in a clockwise direction.

Here's an example of a left-left rotation:

Before rotation:

```

    4
   /\
  3 5
 /\  \
2   6

/

1

```



After left-left rotation:



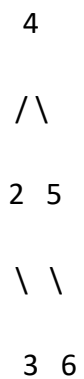
#### LEFT-RIGHT ROTATION:

This rotation is used when the left subtree of the right child of a node becomes taller by two levels compared to the right subtree.

To restore balance, a left-right rotation is performed by rotating the unbalanced node's right child to the right, followed by a left-left rotation.

Here's an example of a left-right rotation:

Before rotation:







After left-right rotation:

```
    4
   /\
  3 5
 /\ 
2  6
```

#### RIGHT-RIGHT ROTATION:

This rotation is used when the right subtree of the right child of a node becomes taller by two levels compared to the left subtree.

To restore balance, a right-right rotation is performed by rotating the unbalanced node and its right child in an anti-clockwise direction.

Here's an example of a right-right rotation:

Before rotation:

```
    4
   /\
  2 5
    \
    6
```

After right-right rotation:

```
    5
   /\
```



4 6

/ \

2 3

#### RIGHT-LEFT ROTATION:

This rotation is used when the right subtree of the left child of a node becomes taller by two levels compared to the left subtree.

To restore balance, a right-left rotation is performed by rotating the unbalanced node's left child to the left, followed by a right-right rotation.

Here's an example of a right-left rotation:

Before rotation:

4

/ \

3 6

/ /

2 5

After right-left rotation:

4

/ \

5 6

/ \

3 2



These rotation operations are crucial for maintaining the balance of an AVL tree. They adjust the tree structure while preserving the binary search property, allowing efficient operations on the AVL tree.

## 1. INSERTION:

Insertion in an AVL tree starts like regular BST insertion, by comparing the value to be inserted with the values in the tree and finding the appropriate position.

Once the new node is inserted, the tree might become unbalanced, violating the height balance property.

To restore the balance, we check the balance factor (height difference between left and right subtrees) of the ancestors of the inserted node.

If a node becomes unbalanced, there are four cases: left-left, left-right, right-right, and right-left rotations.

Here's an example of inserting the values 5, 3, 7, 1, and 4 into an empty AVL tree:

```
5
 / \
3   7
 / \
1   4
```

After insertion, the tree remains balanced with a maximum height difference of 1.

## 2. SEARCHING:

Searching in an AVL tree is the same as searching in a regular BST.



We compare the search value with the values in the tree and traverse left or right until we find a match or reach a null pointer.

The AVL tree's balanced structure does not affect the searching operation.

Here's an example of searching for the value 4 in the AVL tree:

```
5
 / \
3   7
 / \
1   4
```

The search path is the same as in a regular BST:  $5 \rightarrow 3 \rightarrow 4$  (found).

### 3. DELETION:

Deletion in an AVL tree follows the same steps as deletion in a regular BST.

Once a node is deleted, the tree might become unbalanced. Similar to insertion, we check the balance factor of the ancestors of the deleted node. If a node becomes unbalanced, rotations are performed to restore the balance.

Here's an example of deleting the value 3 from the AVL tree:

```
5
 / \
3   7
 / \
1   4
```

After deletion, the tree remains balanced:

```
5
```



/\  
4 7  
/  
1

The key distinction of AVL trees from regular BSTs is the maintenance of balance through rotations during insertion and deletion operations. These rotations ensure that the AVL tree maintains its height balance property, leading to efficient searching, insertion, and deletion operations.

The balanced structure of AVL trees provides guarantees on the worst-case time complexity of these operations, making them suitable for scenarios where balanced trees are essential.



---

## KEY TAKEAWAYS

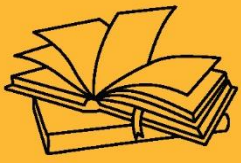
- **Self-Balancing Property:** AVL trees are self-balancing binary search trees that maintain a balanced structure by enforcing a height balance property. This property ensures that the heights of the left and right subtrees of any node differ by at most one.
- **Insertion:** When inserting a new value into an AVL tree, the regular BST insertion process is followed. After insertion, the tree may become unbalanced. To restore balance, rotations are performed based on the specific imbalance case. These rotations maintain the height balance property of the AVL tree.
- **Searching:** Searching in an AVL tree is the same as in a regular BST. The balanced structure of the AVL tree does not affect the searching operation. The search process involves comparing the search value with the values in the tree and traversing left or right based on the comparison.
- **Deletion:** Deletion in an AVL tree follows the regular BST deletion process. After deletion, the tree may become unbalanced. Similar to insertion, rotations are performed to restore balance by considering the imbalance cases. These rotations ensure that the height balance property is maintained in the AVL tree.
- **Efficiency:** The balancing property of AVL trees guarantees efficient searching, insertion, and deletion operations. The height balance property keeps the height of the tree logarithmic, providing a balanced structure and reducing the worst-case time complexity of operations.
- **AVL Tree Rotations:** The balancing of AVL trees is achieved through four types of rotations: left-left, left-right, right-right, and right-left rotations. These rotations preserve the ordering of the BST while adjusting the tree structure to maintain balance.



---

## OUTCOME OF CHAPTER 7

- Describe tree structures and terminology (nodes, levels, height, etc.).
- Differentiate between types: Binary Tree, BST, AVL, B-Tree.
- Perform and analyze insertion, deletion, and traversal (inorder, preorder, postorder).
- Implement tree structures in code and handle balancing.
- Apply tree-based logic in hierarchical storage and quick search systems.
- Understand applications in databases, file systems, and decision-making models.



## NON-LINEAR DATA STRUCTURE







## SUB LESSON 8.1

### BASIC TERMINOLOGIES OF GRAPH

#### OBJECTIVES OF CHAPTER 8

- Understand graph terminology and representation (adjacency list/matrix).
- Study types of graphs: directed, undirected, weighted, and unweighted.
- Implement BFS and DFS traversal techniques.
- Explore minimum spanning tree algorithms: Prim's and Kruskal's.
- Apply graph algorithms in networking, pathfinding, and optimization.

#### BASIC TERMINOLOGIES OF GRAPH

Graph data structure terminology refers to the fundamental concepts and terms used when discussing graphs. Here are some basic terminologies of graph data structure:

**Graph:** A graph is a non-linear data structure consisting of a set of vertices (also called nodes) connected by edges. It represents a collection of interconnected elements.

**Vertex/Node:** A vertex or node is a fundamental unit of a graph. It represents a discrete entity in the graph, such as a person, object, or location. Each vertex can have additional attributes or properties associated with it.

**Edge:** An edge is a connection between two vertices in a graph. It represents a relationship or link between the associated entities. An edge may be directed (one-way) or undirected (two-way).

**Directed Graph:** A directed graph, or digraph, is a graph in which edges have a specific direction. The edges indicate a one-way relationship between the vertices. If there is an edge from vertex A to vertex B, it does not imply that there is an edge from B to A.



**Undirected Graph:** An undirected graph is a graph in which edges have no specific direction. The edges represent a symmetric relationship between the vertices. If there is an edge between vertex A and vertex B, it implies that there is an edge between B and A as well.

**Weighted Graph:** A weighted graph is a graph in which each edge is assigned a weight or cost. The weight represents the value or distance associated with the edge, indicating the strength of the relationship or the cost of traversing that edge.

**Degree:** The degree of a vertex in a graph refers to the number of edges connected to that vertex. In an undirected graph, the degree counts both incoming and outgoing edges. In a directed graph, the degree can be divided into in-degree (number of incoming edges) and out-degree (number of outgoing edges).

**Path:** A path in a graph is a sequence of vertices connected by edges. It represents a route or a sequence of steps to traverse from one vertex to another. The length of a path can be measured by the number of edges or the sum of edge weights.

**Cycle:** A cycle in a graph is a path that starts and ends at the same vertex, forming a closed loop. It involves traversing through multiple vertices and edges without repeating any vertex (except the starting and ending vertex).

**Connected Graph:** A connected graph is a graph in which there is a path between any two vertices. There are no isolated or disconnected vertices.

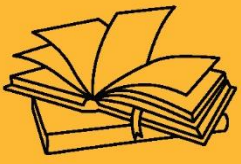
These are some of the basic terminologies used in graph data structure. Understanding these concepts is essential when working with graphs and graph algorithms



---

## KEY TAKEAWAYS

- Linked lists are data structures consisting of nodes with data and references to the next node.
- Graphs consist of vertices (nodes) connected by edges, representing a collection of interconnected elements.
- Vertices represent discrete entities, and edges represent relationships or links between the entities.
- Graphs can be either directed (edges have a specific direction) or undirected (edges have no specific direction).
- Weighted graphs assign values or costs to the edges, indicating the strength of the relationship or the cost of traversal.
- The degree of a vertex refers to the number of edges connected to it, and it can be divided into in-degree and out-degree in a directed graph.
- Paths represent routes or sequences of vertices and edges to traverse from one vertex to another.
- Cycles are closed paths that start and end at the same vertex, without repeating any other vertex (except the starting and ending vertex).
- Connected graphs have a path between any two vertices, with no isolated or disconnected vertices.
- By understanding these terminologies, you'll have a solid foundation for working with graph data structures, implementing graph algorithms, and analyzing relationships and connectivity within a graph.



## NON-LINEAR DATA STRUCTURE





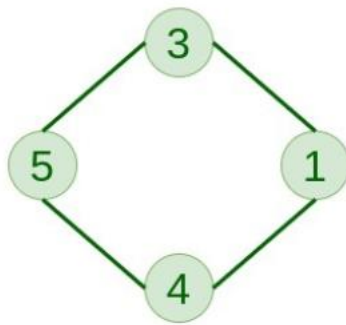
## SUB LESSON 8.2

### TYPES OF GRAPH

#### TYPES OF GRAPH

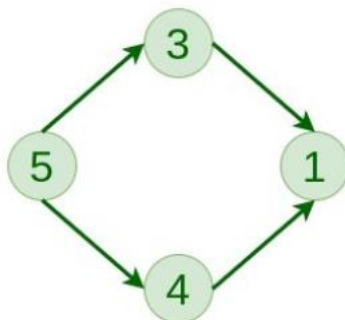
##### UNDIRECTED GRAPH:

In an undirected graph, edges have no specific direction. The relationship between vertices is symmetric. An undirected graph is represented by lines connecting the vertices.



##### DIRECTED GRAPH (DIGRAPH):

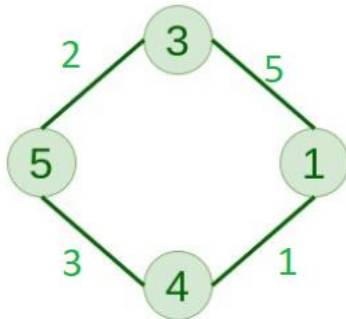
In a directed graph, edges have a specific direction. The relationship between vertices is asymmetric. A directed graph is represented by arrows indicating the direction of the edge.



##### WEIGHTED GRAPH:

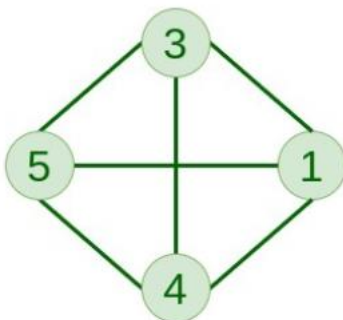


In a weighted graph, edges have weights or costs associated with them. These weights represent the value or distance between vertices. The weights are typically represented next to the edges.



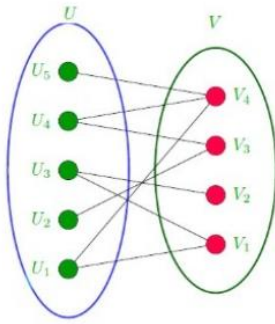
#### COMPLETE GRAPH:

In a complete graph, every pair of vertices is connected by an edge. It is fully connected, meaning there is an edge between every pair of vertices.



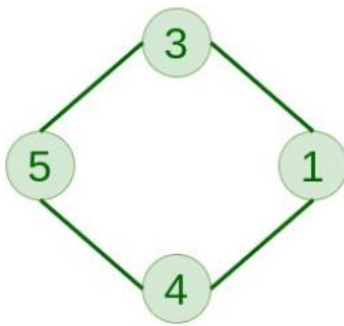
#### BIPARTITE GRAPH:

In a bipartite graph, the vertices can be divided into two disjoint sets, such that every edge connects a vertex from one set to a vertex from the other set. There are no edges within the same set.



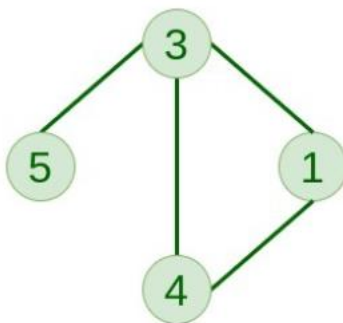
### CYCLIC GRAPH:

In a cyclic graph, there is at least one cycle present, which is a closed path that starts and ends at the same vertex.



### ACYCLIC GRAPH:

In an acyclic graph, there are no cycles present. All paths are non-repetitive and do not form closed loops.





These diagrams illustrate the visual representation of each graph type, helping to understand the structural characteristics of different graphs.

---

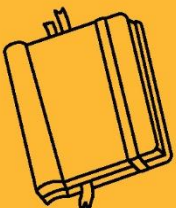
### KEY TAKEAWAYS

- **Undirected Graph:** In an undirected graph, edges have no direction, and the relationship between vertices is symmetric.
- **Directed Graph (Digraph):** In a directed graph, edges have a specific direction, and the relationship between vertices is asymmetric.
- **Weighted Graph:** In a weighted graph, edges have weights or costs associated with them, representing the value or distance between vertices.
- **Complete Graph:** In a complete graph, every pair of vertices is connected by an edge, resulting in a fully connected graph.
- **Bipartite Graph:** In a bipartite graph, the vertices can be divided into two disjoint sets, with edges connecting vertices from one set to the other.
- **Cyclic Graph:** A cyclic graph contains at least one cycle, which is a closed path that starts and ends at the same vertex.
- **Acyclic Graph:** An acyclic graph has no cycles, and all paths are non-repetitive, forming no closed loops.
- Understanding these types of graphs helps in analyzing and solving various graph-related problems. It provides insights into the nature of relationships, connectivity, and structural properties of graphs, enabling the selection of appropriate algorithms and techniques for graph manipulation and analysis.





## **NON-LINEAR DATA STRUCTURE**





## SUB LESSON 8.3

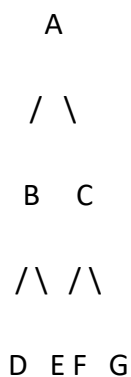
### BREADTH FIRST SEARCH

#### BREDTH FIRST SEARCH

BFS (Breadth-First Search) is a graph traversal algorithm that explores all the vertices of a graph in breadth-first order, i.e., it visits all the vertices at the same level before moving on to the vertices at the next level. The algorithm starts at a given source vertex and systematically explores its neighboring vertices before moving to the next level.

Here's an explanation of the BFS algorithm with an example:

Consider the following undirected graph:



Let's say we want to perform a BFS starting from vertex A.

Start by marking vertex A as visited and enqueue it in a queue (or a FIFO data structure) to keep track of the vertices to be visited.

Visited: [A]

Queue: [A]

Dequeue the first vertex from the queue, which is A.

Visited: [A]

Queue: []

Explore the neighboring vertices of A (B and C) and enqueue them if they are not already visited.



Visited: [A]

Queue: [B, C]

Dequeue the next vertex from the queue, which is B.

Visited: [A, B]

Queue: [C]

Explore the neighboring vertices of B (D and E) and enqueue them if they are not already visited.

Visited: [A, B]

Queue: [C, D, E]

Dequeue the next vertex from the queue, which is C.

Visited: [A, B, C]

Queue: [D, E]

Explore the neighboring vertices of C (F and G) and enqueue them if they are not already visited.

Visited: [A, B, C]

Queue: [D, E, F, G]

Dequeue the next vertex from the queue, which is D.

Visited: [A, B, C, D]

Queue: [E, F, G]

Dequeue the next vertex from the queue, which is E.

Visited: [A, B, C, D, E]

Queue: [F, G]

Dequeue the next vertex from the queue, which is F.

Visited: [A, B, C, D, E, F]

Queue: [G]

Dequeue the last vertex from the queue, which is G.

Visited: [A, B, C, D, E, F, G]

Queue: []

The queue is empty, indicating that all vertices have been visited.



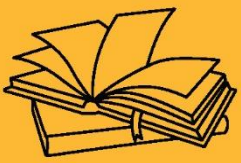
The BFS algorithm visits the vertices in the order A, B, C, D, E, F, G, which represents the breadth-first exploration of the graph starting from vertex A.

BFS is useful for various graph-related tasks such as finding the shortest path between two vertices, finding connected components, or checking for bipartiteness. Its breadth-first nature ensures that the shortest paths are discovered first as it explores all the neighboring vertices at the current level before moving deeper into the graph.

---

### KEY TAKEAWAYS

- BFS explores the vertices of a graph in breadth-first order, visiting all vertices at the same level before moving to the next level.
- The algorithm starts at a given source vertex and systematically explores its neighboring vertices.
- A queue (or a FIFO data structure) is used to keep track of the vertices to be visited.
- The algorithm marks each visited vertex to avoid revisiting it.
- BFS ensures that the shortest paths are discovered first as it explores all neighboring vertices at the current level before moving deeper into the graph.
- BFS can be used for various graph-related tasks, such as finding the shortest path between two vertices, determining connectivity, or checking for bipartiteness.
- The order of visiting vertices in BFS depends on the structure of the graph and the starting vertex.
- BFS guarantees that all vertices reachable from the source vertex are visited.



## **NON-LINEAR DATA STRUCTURE**





## SUB LESSON 8.4

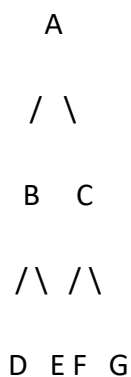
### DEPTH FIRST SEARCH

#### DEPTH FIRST SEARCH

DFS (Depth-First Search) is a graph traversal algorithm that explores as far as possible along each branch before backtracking. It starts at a given source vertex and explores its adjacent vertices recursively, moving deeper into the graph until no more unvisited vertices are reachable. Once it reaches a dead end, it backtracks and continues exploring other branches.

Here's an explanation of the DFS algorithm with an example:

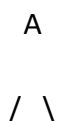
Consider the following undirected graph:



DFS (Depth-First Search) is a graph traversal algorithm that explores as far as possible along each branch before backtracking. It starts at a given source vertex and explores its adjacent vertices recursively, moving deeper into the graph until no more unvisited vertices are reachable. Once it reaches a dead end, it backtracks and continues exploring other branches.

Here's an explanation of the DFS algorithm with an example:

Consider the following undirected graph





B C

/\ /\

D E F G

Let's say we want to perform a DFS starting from vertex A.

Start by marking vertex A as visited.

Visited: [A]

Explore one of the adjacent vertices of A (B) that is not visited.

Visited: [A, B]

Continue exploring deeper into the graph from vertex B. Explore one of its adjacent vertices (D) that is not visited.

Visited: [A, B, D]

Since vertex D has no unvisited adjacent vertices, backtrack to vertex B and explore its other adjacent vertex (E) that is not visited.

Visited: [A, B, D, E]

Again, backtrack to vertex B and explore its other adjacent vertex (C) that is not visited.

Visited: [A, B, D, E, C]

Explore deeper into the graph from vertex C. Explore one of its adjacent vertices (F) that is not visited.

Visited: [A, B, D, E, C, F]

Since vertex F has no unvisited adjacent vertices, backtrack to vertex C and explore its other adjacent vertex (G) that is not visited.

Visited: [A, B, D, E, C, F, G]

Since all vertices have been visited, the algorithm terminates.

The DFS algorithm visits the vertices in the order A, B, D, E, C, F, G, which represents the depth-first exploration of the graph starting from vertex A.



DFS is useful for various graph-related tasks such as finding connected components, determining cycles, or searching for paths. Its depth-first nature allows for systematic exploration of the graph, deeply exploring one branch at a time, before backtracking to explore other branches.

---

### KEY TAKEAWAYS

- Depth first search (DFS) is a graph traversal algorithm that explores the nodes of a graph as far as possible along each branch before backtracking. It uses a stack data structure to keep track of the visited and unvisited nodes. DFS can be implemented recursively or iteratively. Some key takeaways of DFS are:
- DFS can find a solution without examining much of the search space if the solution is close to the root node.
- DFS requires less memory than breadth first search (BFS) since it only stores the nodes on the current path.
- DFS can be used to find cycles, strongly connected components, topological sorting, and other properties of graphs.
- DFS is not guaranteed to find the shortest path or the optimal solution, unlike BFS.
- DFS can get stuck in infinite loops if the graph is not finite or has cycles.





## **NON-LINEAR DATA STRUCTURE**





## SUB LESSON 8.5

### PRIMS ALGORITHM

#### PRIMS ALGORITHM

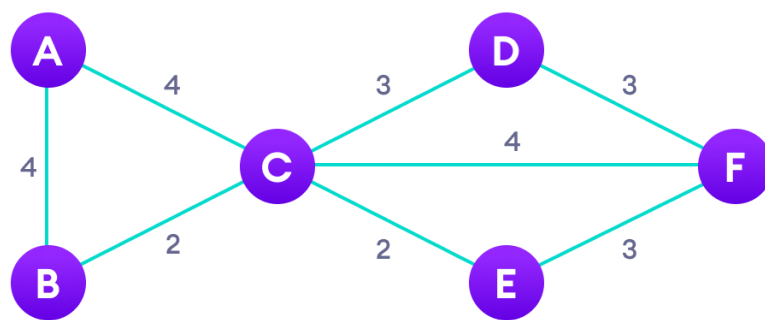
Prim's algorithm is a greedy algorithm used to find a minimum spanning tree (MST) in a weighted undirected graph. The algorithm starts with a single vertex and grows the MST by adding edges that connect the growing tree to new vertices, always selecting the minimum-weight edge that connects the tree to an unvisited vertex. Here's how Prim's algorithm works in detail:

1. Initialize an empty MST and a set of visited vertices.
2. Choose any vertex to start the algorithm. Add it to the MST and mark it as visited.
3. While there are unvisited vertices:
  - a. Consider all edges that connect the visited vertices to the unvisited vertices.
  - b. Choose the edge with the minimum weight.
  - c. Add the selected edge to the MST and mark the corresponding unvisited vertex as visited.
4. Repeat step 3 until all vertices are visited.

Illustration of the steps of Prim's algorithm is as given below:

1. Start with a weighted graph
2. Choose a vertex
3. Choose the shortest edge from this vertex and add it
4. Choose the nearest vertex not yet in the solution
5. Choose the nearest edge not yet in the solution, if there are multiple choices, choose one at random
6. Repeat until you have a spanning tree

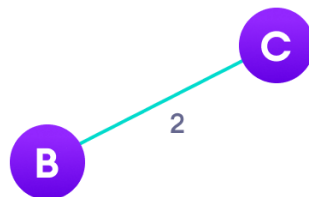
Consider the following undirected weighted graph:



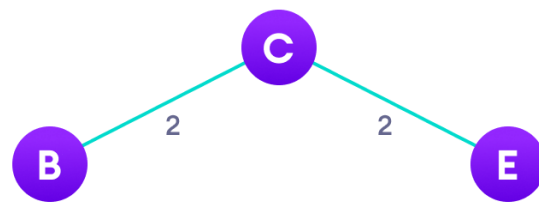
Step: 1



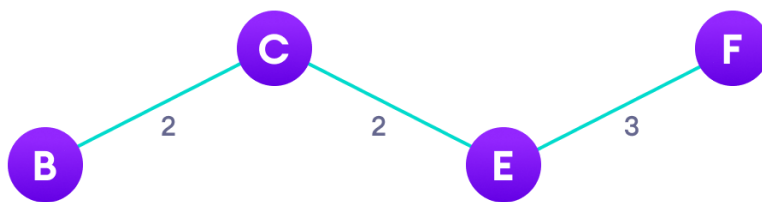
Step: 2



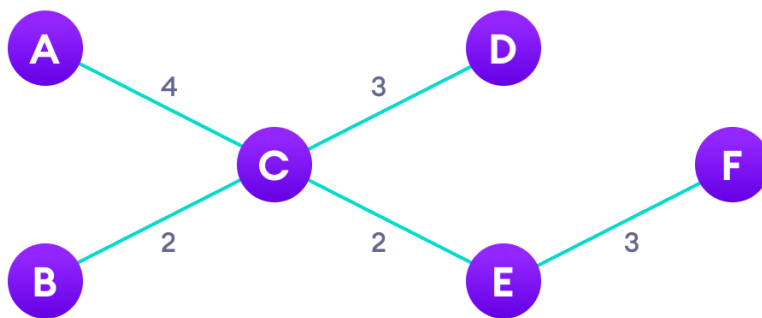
Step: 3



Step: 4



Step: 5



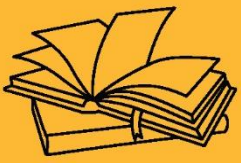
Step: 6



---

## KEY TAKEAWAYS

- Prim's algorithm is a greedy algorithm used to find a minimum spanning tree (MST) in a weighted undirected graph.
- The algorithm starts with a single vertex and grows the MST by adding edges that connect the growing tree to new vertices.
- At each step, Prim's algorithm selects the minimum-weight edge that connects the tree to an unvisited vertex.
- The algorithm continues until all vertices are visited, resulting in a connected, acyclic subgraph that spans all vertices with the minimum total weight.
- Prim's algorithm ensures that the MST is obtained, which is the subgraph with the minimum total weight among all possible spanning trees.
- The complexity of Prim's algorithm is  $O(V^2)$  for adjacency matrix representation and  $O(E \log V)$  for adjacency list representation, where  $V$  is the number of vertices and  $E$  is the number of edges in the graph.
- Prim's algorithm is commonly used in network design, cluster analysis, and other applications where finding the minimum cost spanning tree is essential.



## **NON-LINEAR DATA STRUCTURE**





## SUB LESSON 8.6

### KRUSKAL ALGORITHM

#### KRUSKAL ALGORITHM

The Kruskal's algorithm is a widely used algorithm in graph theory that finds a minimum spanning tree (MST) for a connected, weighted graph. The MST is a subgraph that connects all the vertices of the original graph with the minimum total weight.

Here's a step-by-step explanation of the Kruskal's algorithm:

It falls under a class of algorithms called greedy algorithms that find the local optimum in the hopes of finding a global optimum.

We start from the edges with the lowest weight and keep adding edges until we reach our goal.

The steps for implementing Kruskal's algorithm are as follows:

1. Sort all the edges from low weight to high
2. Take the edge with the lowest weight and add it to the spanning tree. If adding the edge created a cycle, then reject this edge.
3. Keep adding edges until we reach all vertices.

By selecting the edges with the minimum weight and ensuring there are no cycles, the Kruskal's algorithm gradually builds the MST, guaranteeing that the total weight of the MST is minimized.

It's worth noting that Kruskal's algorithm is a greedy algorithm, as it makes locally optimal choices at each step to construct the final solution.

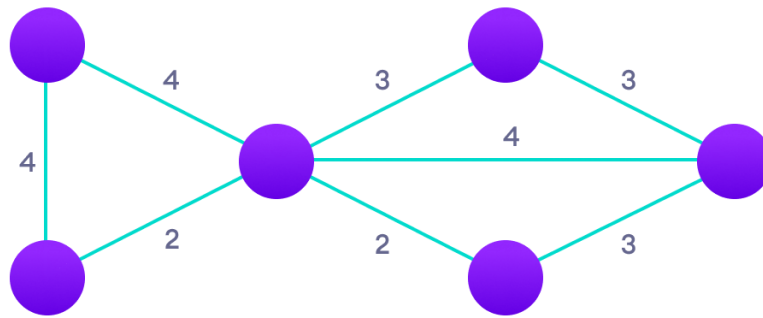
Example of Kruskal's algorithm:

Consider the following undirected graph with its corresponding weights:

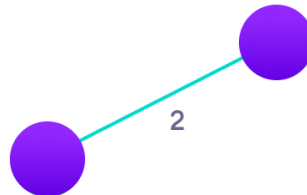
1. Start with a weighted graph
2. Choose the edge with the least weight, if there are more than 1, choose anyone



3. Choose the next shortest edge and add it
4. Choose the next shortest edge that doesn't create a cycle and add it
5. Choose the next shortest edge that doesn't create a cycle and add it
6. Repeat until you have a spanning tree

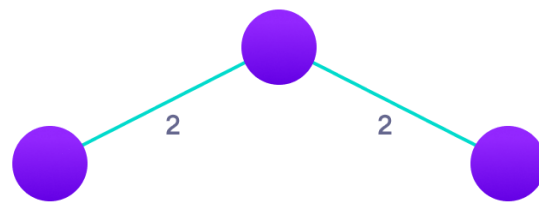


Step: 1

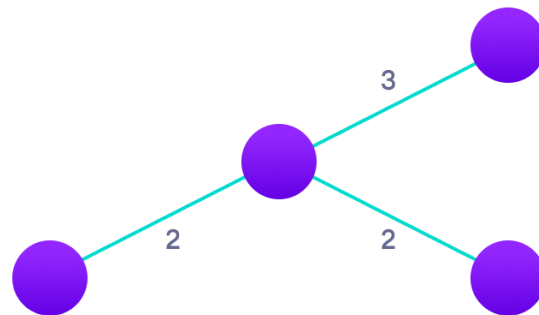


Step: 2

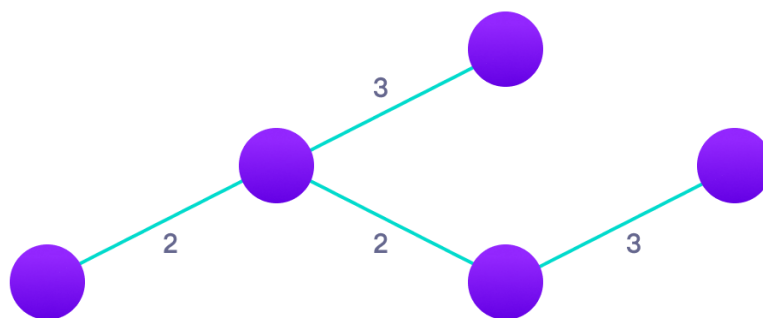




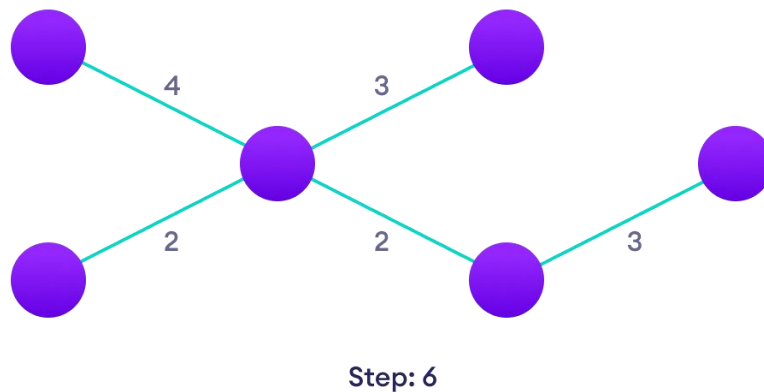
Step: 3



Step: 4



Step: 5



### KEY TAKEAWAYS

- It is a greedy algorithm: Kruskal's algorithm makes locally optimal choices at each step by selecting the edges with the minimum weight.
- Sorting the edges: The algorithm starts by sorting all the edges of the graph based on their weights in ascending order.
- Checking for cycles: Kruskal's algorithm uses a disjoint-set data structure (such as Union-Find) to check if adding an edge to the MST creates a cycle. This step ensures that the resulting MST is acyclic.
- Adding edges to the MST: The algorithm iteratively selects edges from the sorted list, checks for cycles, and adds the edge to the MST if it does not create a cycle.
- Terminating condition: The algorithm stops when the MST contains  $V-1$  edges, where  $V$  is the number of vertices in the original graph
- Remember that the MST found by Kruskal's algorithm is not unique, but all valid MSTs will have the same total weight.



---

## OUTCOMES OF CHAPTER 8

- Understand graph representation and terminology (nodes, edges, degree).
- Implement graphs using adjacency matrix and list methods.
- Apply BFS and DFS for traversal and pathfinding.
- Use Prim's and Kruskal's algorithms to find minimum spanning trees.
- Solve real-world problems involving networks, maps, and connectivity.
- Analyze and choose the right graph algorithm for different applications.